



# **POLITECNICO**

## **MILANO 1863**

School of Industrial and Information Engineering

High Performance Computing Engineering

### **Numerical Methods for Partial Differential Equations**

#### **Navier-Stokes equations problems: A Finite Element Solver using Deal.II**

Academic Year 2025/2026

#### **Team Members**

| <b>Person code</b> | <b>Surname</b> | <b>Name</b> |
|--------------------|----------------|-------------|
| 10822323           | Nonino         | Chiara      |
| 10896741           | Tinaoui        | Alessandro  |
| 10819786           | Vaccaro        | Giovanni    |

---

# Contents

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>1</b>  |
| <b>2</b> | <b>Navier-Stokes equations</b>                      | <b>1</b>  |
| <b>3</b> | <b>Mathematical model</b>                           | <b>1</b>  |
| 3.1      | Weak Formulation . . . . .                          | 2         |
| 3.1.1    | Wellposedness, Stability and Convergence . . . . .  | 2         |
| 3.2      | Semi-discrete Formulation . . . . .                 | 3         |
| 3.3      | Fully-discrete Formulation . . . . .                | 4         |
| 3.4      | Algebraic System . . . . .                          | 5         |
| 3.5      | Stabilized Formulation . . . . .                    | 6         |
| 3.5.1    | Temam stabilization . . . . .                       | 6         |
| 3.5.2    | Grad-div stabilization . . . . .                    | 7         |
| 3.5.3    | SUPG stabilization . . . . .                        | 8         |
| <b>4</b> | <b>Finite Element Implementation</b>                | <b>9</b>  |
| 4.1      | Mesh Generation . . . . .                           | 9         |
| 4.2      | Finite Element Spaces . . . . .                     | 11        |
| 4.3      | Boundary Conditions . . . . .                       | 11        |
| 4.4      | Assembly of the System . . . . .                    | 13        |
| 4.4.1    | Assembly of static matrices . . . . .               | 13        |
| 4.4.2    | Assembly of time-dependent matrices . . . . .       | 14        |
| 4.5      | Force coefficients and pressure probes . . . . .    | 15        |
| <b>5</b> | <b>Preconditioners</b>                              | <b>17</b> |
| 5.1      | Block factorization and Schur complement . . . . .  | 17        |
| 5.2      | SIMPLE Preconditioner . . . . .                     | 17        |
| 5.3      | Block Triangular Preconditioner . . . . .           | 18        |
| 5.4      | Yosida Preconditioner . . . . .                     | 19        |
| 5.5      | PCD Preconditioner . . . . .                        | 19        |
| <b>6</b> | <b>Results and Validation</b>                       | <b>21</b> |
| 6.1      | Physical Validation Benchmarks . . . . .            | 21        |
| 6.1.1    | V1 – 2D-1 Steady Validation . . . . .               | 22        |
| 6.1.2    | V2 – 2D-2 Periodic Validation . . . . .             | 23        |
| 6.1.3    | V3 – 2D-3 Ramp Validation on L0 . . . . .           | 24        |
| 6.1.4    | V4 – Mesh Refinement on 2D-3 . . . . .              | 24        |
| 6.1.5    | V5 – Timestep Refinement on 2D-3 . . . . .          | 25        |
| 6.1.6    | V6 – Reynolds Sweep on 2D-3 . . . . .               | 26        |
| 6.1.7    | V7 – Grad-Div Stabilization Sweep . . . . .         | 27        |
| 6.1.8    | V8 – 3D Validation . . . . .                        | 28        |
| 6.2      | Preconditioner and Performance Benchmarks . . . . . | 29        |
| 6.2.1    | P0 – Inner Max-Iteration Tuning . . . . .           | 29        |
| 6.2.2    | P1 – Baseline Preconditioner Comparison . . . . .   | 30        |
| 6.2.3    | P2 – Timestep Sensitivity . . . . .                 | 31        |
| 6.2.4    | P3 – Mesh Sensitivity and Optimality . . . . .      | 31        |
| 6.2.5    | P4 – Reynolds and Viscosity Sensitivity . . . . .   | 32        |

---

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 6.2.6    | P5 – SIMPLE Relaxation Sweep . . . . . | 32        |
| 6.2.7    | P6 – Cost Breakdown . . . . .          | 33        |
| 6.2.8    | P7 – MPI Strong Scaling . . . . .      | 33        |
| 6.3      | Summary of Validation Status . . . . . | 33        |
| <b>7</b> | <b>Conclusions</b>                     | <b>34</b> |
|          | <b>References</b>                      | <b>34</b> |

---

# 1 Introduction

The numerical approximation of viscous incompressible flow is a central problem in computational fluid dynamics. This project presents the implementation and numerical investigation of an unsteady incompressible Navier–Stokes solver using the finite element method to simulate the benchmark problem “*Flow past a cylinder*” in 2D and 3D environments. The numerical methods employed, along with their stability and accuracy, are discussed throughout the report. In order to improve computational efficiency, preconditioning strategies are adopted for high performance computing applications.

The focus of this work lies in the aerodynamic characterization of the flow for different values of the Reynolds number, with particular attention to the time evolution of the lift and drag coefficients,  $C_L$  and  $C_D$ , which serve as the primary indicators of the forces exerted by the fluid on the cylindrical obstacle in the directions perpendicular and parallel to the flow, respectively.

## 2 Navier-Stokes equations

The unsteady, incompressible viscous fluid can be described by the Navier-Stokes equations. Given a spatial domain  $\Omega \subset \mathbb{R}^d$  (with  $d = 2, 3$ ) and a time interval  $(0, T]$ , the system of equations for the velocity field  $\mathbf{u}(\mathbf{x}, t)$  and the pressure field  $p(\mathbf{x}, t)$  is given by:

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} - \nu \Delta \mathbf{u} + \nabla p = f & \text{in } \Omega, \\ \nabla \cdot \mathbf{u} = 0 & \text{in } \Omega, \\ \mathbf{u} = \mathbf{g} & \text{on } \Gamma_D \subset \partial\Omega, \\ \nu \nabla \mathbf{u} \mathbf{n} - p \mathbf{n} = \mathbf{h} & \text{on } \Gamma_N = \partial\Omega \setminus \Gamma_D, \\ \mathbf{u}(t = 0) = \mathbf{u}_0 & \text{in } \Omega \end{cases} \quad (1)$$

In this formulation,  $\nu > 0$  expresses the kinematic viscosity of the fluid and  $f$  represents the forcing term. The first equation denotes the momentum equation including acceleration of the fluid, non linear convection term and the viscous term. The second equation instead, represents the continuity equation ensuring the incompressibility of the fluid.

The boundary  $\partial\Omega$  is partitioned into two disjoint subsets, such that  $\Gamma_D \cup \Gamma_N = \partial\Omega$  and  $\Gamma_D \cap \Gamma_N = \emptyset$ , where  $\Gamma_D$  is the Dirichlet boundary condition and  $\Gamma_N$  the Neumann boundary condition, applied via  $\mathbf{g}$  and  $\mathbf{h}$  functions directly at the boundary, respectively setting velocity of the fluid and the natural boundary traction. The last term is the initial condition for the velocity field, which must satisfy the incompressibility condition  $\nabla \cdot \mathbf{u}_0 = 0$ .

## 3 Mathematical model

In order to solve the Navier-Stokes equations numerically, we employ a Finite Element Method (FEM) for spatial discretization combined with a one-step  $\theta$ -method for time integration.

### 3.1 Weak Formulation

Introducing the functional spaces for velocity and pressure, we define the homogeneous velocity space

$$V_0 = \{ \mathbf{v} \in [H^1(\Omega)]^d : \mathbf{v} = \mathbf{0} \text{ on } \Gamma_D \},$$

and the pressure space

$$Q = L^2(\Omega).$$

Since the Dirichlet boundary condition is in general non-homogeneous,  $\mathbf{u} = \mathbf{g}$  on  $\Gamma_D$ , we introduce a lifting function  $R(\mathbf{g}) \in [H^1(\Omega)]^d$  such that

$$R(\mathbf{g})|_{\Gamma_D} = \mathbf{g}, \quad R(\mathbf{g})|_{\Gamma_N} = \mathbf{0}.$$

The velocity field is decomposed as

$$\mathbf{u} = \mathbf{u}_0 + R(\mathbf{g}),$$

where  $\mathbf{u}_0 \in V_0$  satisfies homogeneous Dirichlet boundary conditions.

Multiplying the momentum equation by a test function  $\mathbf{v} \in V_0$  and the continuity equation by  $q \in Q$ , then integrating by parts the viscous and pressure terms over  $\Omega$ , the weak formulation with lifting reads:

$$\left\{ \begin{array}{l} \forall t > 0, \text{ find } (\mathbf{u}_0(t), p(t)) \in V_0 \times Q \text{ such that, } \forall (\mathbf{v}, q) \in V_0 \times Q, \\ \left( \frac{\partial \mathbf{u}_0}{\partial t}, \mathbf{v} \right) + a(\mathbf{u}_0, \mathbf{v}) + c(\mathbf{u}_0 + R(\mathbf{g}), \mathbf{u}_0 + R(\mathbf{g}), \mathbf{v}) - b(p, \mathbf{v}) = F_R(\mathbf{v}), \\ b(q, \mathbf{u}_0) = -b(q, R(\mathbf{g})). \end{array} \right. \quad (1)$$

where the bilinear and trilinear forms are defined as:

- $a(\mathbf{u}, \mathbf{v}) = \int_{\Omega} \nu \nabla \mathbf{u} : \nabla \mathbf{v} \, d\Omega$
- $c(\mathbf{w}, \mathbf{u}, \mathbf{v}) = \int_{\Omega} ((\mathbf{w} \cdot \nabla) \mathbf{u}) \cdot \mathbf{v} \, d\Omega$
- $b(q, \mathbf{v}) = \int_{\Omega} q(\nabla \cdot \mathbf{v}) \, d\Omega$

The linear functional  $F_R(\mathbf{v})$  accounts for the external forces  $\mathbf{f}$  and the Neumann boundary conditions  $\mathbf{h}$  on  $\Gamma_N$  and the lifting part:

$$\begin{aligned} F_R(\mathbf{v}) = & \int_{\Omega} \mathbf{f} \cdot \mathbf{v} \, d\Omega + \int_{\Gamma_N} \mathbf{h} \cdot \mathbf{v} \, d\gamma \\ & - \left( \frac{\partial R(\mathbf{g})}{\partial t}, \mathbf{v} \right) - a(R(\mathbf{g}), \mathbf{v}). \end{aligned} \quad (2)$$

#### 3.1.1 Wellposedness, Stability and Convergence

The well-posedness of the weak formulation relies on the stability of the velocity–pressure coupling and on the properties of the nonlinear Navier–Stokes operator. For the steady problem, existence of weak solutions can be established under standard assumptions, whereas uniqueness is generally guaranteed only for sufficiently small Reynolds numbers

(or, equivalently, sufficiently small forcing data). The mixed formulation requires the velocity and pressure spaces to satisfy the Ladyzhenskaya–Babuška–Brezzi (LBB) condition

$$\inf_{q \in Q \setminus \{0\}} \sup_{\mathbf{v} \in V_0 \setminus \{0\}} \frac{b(q, \mathbf{v})}{\|\mathbf{v}\|_{H^1(\Omega)} \|q\|_{L^2(\Omega)}} \geq \beta > 0 \quad (3)$$

which guarantees stability of the saddle-point problem and ensures uniqueness of the pressure up to an additive constant. In practice, this indeterminacy is removed by imposing an additional pressure normalisation condition.

At the discrete level, the finite-dimensional spaces  $(V_h, Q_h)$  must inherit the inf-sup stability of the continuous formulation. Failure to satisfy the discrete LBB condition results in spurious pressure modes and a badly-conditioned algebraic system. The Taylor–Hood finite elements adopted in this work, consisting of continuous piecewise polynomials of degree  $r_u$  for the velocity field and degree  $r_p = r_u - 1$  for the pressure field, satisfy a discrete inf-sup condition uniformly with respect to the mesh size  $h$ . Consequently, the Galerkin approximation is stable and convergent. Assuming sufficient regularity of the exact solution, standard mixed finite-element theory yields the optimal error estimates

$$\|u - u_h\|_{H^1(\Omega)} \leq Ch^{r_u}, \quad \|p - p_h\|_{L^2(\Omega)} \leq Ch^{r_p+1}. \quad (4)$$

where  $C$  is independent of the mesh size. These results provide the theoretical foundation for the spatial discretisation employed in the semi-discrete scheme presented in the following section.

### 3.2 Semi-discrete Formulation

In order to move the problem from a continuous weak formulation to a computational problem the first step is to introduce a spatial discretization based on the Finite Element Method. The space of finite elements:

$$[X_h^r]^d = \{\mathbf{v}_h \in [C^0(\Omega)]^d : \mathbf{v}_h|_{\mathcal{K}_i} \in [\mathbb{P}^r(\mathcal{K})], \forall \mathcal{K}_i \in \mathcal{T}_h\} \subset H^1(\Omega) \quad (5)$$

where  $\mathcal{T}_h$  is a triangulation of simplicial elements (triangles in 2D),  $d$  is the dimension of the problem,  $h$  denotes the mesh size, usually defined as the maximum diameter of the elements in  $\mathcal{T}_h$  and  $r$  is the polynomial degree used on the finite elements.  $\mathcal{K}$  is the triangular element and  $\mathbb{P}$  constitutes the space of all polynomials of degree  $r$ . The instructed space is contained in the Hilbert space, as required for conformity with the continuous formulation.

We introduce the finite-dimensional homogeneous velocity space:

$$V_{0,h} = \{\mathbf{v}_h \in [X_h^r]^d : \mathbf{v}_h = \mathbf{0} \text{ on } \Gamma_D\} \subset V_0,$$

and the pressure space

$$Q_h = \{q_h \in X_h^{r-1} : q_h|_{\mathcal{K}} \in \mathbb{P}^{r-1}(\mathcal{K}) \quad \forall \mathcal{K} \in \mathcal{T}_h\} \subset L^2(\Omega).$$

with  $N_h = \dim(V_{0,h})$  and  $M_h = \dim(Q_h)$ .

The discrete lifting  $R_h(\mathbf{g}_h)$  is chosen so that  $R_h(\mathbf{g}_h)|_{\Gamma_D} = \mathbf{g}_h$ , and the discrete velocity is written as

$$\mathbf{u}_h = \mathbf{u}_{0,h} + R_h(\mathbf{g}_h).$$

Since the Dirichlet datum is non-homogeneous, the trial and test velocity spaces are different: the discrete velocity belongs to the affine space

$$V_{g,h} = R_h(\mathbf{g}_h) + V_{0,h},$$

whereas the test functions belong to the homogeneous space  $V_{0,h}$ .

In the semi-discrete formulation we write the problem directly in terms of the full discrete velocity  $\mathbf{u}_h \in V_{g,h}$ . For this reason, the lifting terms do not appear explicitly in the semi-discrete equations.

The semi-discrete problem reads:

$$\begin{cases} \forall t > 0, \text{ find } (\mathbf{u}_h(t), p_h(t)) \in V_{g,h} \times Q_h \text{ such that, } \forall (\mathbf{v}_h, q_h) \in V_{0,h} \times Q_h, \\ (\partial_t \mathbf{u}_h, \mathbf{v}_h) + a(\mathbf{u}_h, \mathbf{v}_h) + c(\mathbf{u}_h, \mathbf{u}_h, \mathbf{v}_h) - b(p_h, \mathbf{v}_h) = F_h(\mathbf{v}_h), \\ b(q_h, \mathbf{u}_h) = 0. \end{cases} \quad (6)$$

Where the right-hand side is the physical functional

$$F_h(\mathbf{v}_h) = \int_{\Omega} \mathbf{f} \cdot \mathbf{v}_h d\Omega + \int_{\Gamma_N} \mathbf{h} \cdot \mathbf{v}_h d\gamma.$$

The algebraic unknowns are associated with the free degrees of freedom of the finite element functions. We denote by  $\{\phi_j\}_{j=1}^{N_h}$  the velocity basis associated with  $V_{0,h}$ , and by  $\{\psi_k\}_{k=1}^{M_h}$  the pressure basis associated with  $Q_h$ . The homogeneous part of the velocity and the pressure are expanded as

$$\mathbf{u}_{0,h}(\mathbf{x}, t) = \sum_{j=1}^{N_h} u_j(t) \phi_j(\mathbf{x}), \quad p_h(\mathbf{x}, t) = \sum_{k=1}^{M_h} p_k(t) \psi_k(\mathbf{x}).$$

The full velocity is then recovered as

$$\mathbf{u}_h = \mathbf{u}_{0,h} + R_h(\mathbf{g}_h).$$

### 3.3 Fully-discrete Formulation

Let  $\Delta t > 0$  denote the timestep and let  $t^n = n\Delta t$ . Time integration is performed by means of a one-step  $\theta$ -method, with  $\theta \in [0, 1]$ . Since the convective term is nonlinear, it is linearized by introducing a convective velocity  $\beta$ . Thus, rather than treating  $c(\mathbf{u}_h^{n+1}, \mathbf{u}_h^{n+1}, \mathbf{v}_h)$  directly, we solve a linear problem involving  $c(\beta, \mathbf{u}_h^{n+1}, \mathbf{v}_h)$ .

At each timestep, and at each nonlinear iteration when the Picard strategy is used, the following linearized problem is solved:

$$\begin{cases} \text{Find } (\hat{\mathbf{u}}_h^{n+1,k+1}, \hat{p}_h^{n+1,k+1}) \in V_{g,h} \times Q_h \text{ such that, } \forall (\mathbf{v}_h, q_h) \in V_{0,h} \times Q_h. \\ \frac{1}{\Delta t} (\hat{\mathbf{u}}_h^{n+1,k+1}, \mathbf{v}_h) + \theta a(\hat{\mathbf{u}}_h^{n+1,k+1}, \mathbf{v}_h) + \theta c(\beta, \hat{\mathbf{u}}_h^{n+1,k+1}, \mathbf{v}_h) - b(\hat{p}_h^{n+1,k+1}, \mathbf{v}_h) = G(\mathbf{v}_h), \\ b(q_h, \hat{\mathbf{u}}_h^{n+1,k+1}) = 0. \end{cases} \quad (7)$$

The hatted variables denote the solution of the linearized problem before any relaxation step, with the Dirichlet data imposed at the new time level  $t^{n+1}$ . In the Oseen case

no nonlinear iteration is performed, hence the hatted solution is directly accepted as the solution at the new timestep, namely:

$$\mathbf{u}_h^{n+1} = \widehat{\mathbf{u}}_h^{n+1}, \quad p_h^{n+1} = \widehat{p}_h^{n+1}. \quad (8)$$

In the Picard case, the hatted velocity represents the raw update at nonlinear iteration  $k + 1$  and may be relaxed before being accepted.

The right-hand side contains the explicit part of the  $\theta$ -method and is given by

$$\begin{aligned} G(\mathbf{v}_h) = & \frac{1}{\Delta t}(\mathbf{u}_h^n, \mathbf{v}_h) + \theta(\mathbf{f}^{n+1}, \mathbf{v}_h) + (1 - \theta)(\mathbf{f}^n, \mathbf{v}_h) \\ & - (1 - \theta)a(\mathbf{u}_h^n, \mathbf{v}_h) - (1 - \theta)c(\mathbf{u}_h^n, \mathbf{u}_h^n, \mathbf{v}_h) - \int_{\Gamma_N} p_{\text{out}} \mathbf{n} \cdot \mathbf{v}_h d\sigma. \end{aligned} \quad (9)$$

The choice of  $\beta$  determines the nonlinear strategy. In the Oseen semi-implicit case, only one linear problem is solved per timestep and the convective velocity is taken from the previous timestep:

$$\beta = \mathbf{u}_h^n. \quad (10)$$

In the Picard case, a fixed-point iteration is performed inside each timestep. At nonlinear iteration  $k$ , the convective velocity is the current Picard iterate:

$$\beta = \mathbf{u}_h^{n+1,k}. \quad (11)$$

In the relaxed Picard variant, the accepted iterate is obtained by

$$X^{n+1,k+1} = (1 - \alpha)X^{n+1,k} + \alpha\widehat{X}^{n+1,k+1}, \quad 0 < \alpha \leq 1. \quad (12)$$

Equivalently, both the velocity and pressure blocks are updated by the same relaxation factor in the implementation. The unrelaxed Picard method corresponds to  $\alpha = 1$ . For the Oseen method, the same linearized problem is solved only once per timestep, with  $\beta = \mathbf{u}_h^n$ .

### 3.4 Algebraic System

The non-homogeneous Dirichlet values are imposed strongly at the algebraic level; therefore the block system below is written for the assembled finite element unknowns after applying the boundary constraints:

$$\begin{pmatrix} F(\beta) & -B^T \\ B & 0 \end{pmatrix} \begin{pmatrix} \widehat{\mathbf{U}}^{n+1,k+1} \\ \widehat{\mathbf{P}}^{n+1,k+1} \end{pmatrix} = \begin{pmatrix} \mathbf{G} \\ 0 \end{pmatrix}. \quad (13)$$

Here  $\widehat{\mathbf{U}}^{n+1,k+1} \in \mathbb{R}^{N_h}$  and  $\widehat{\mathbf{P}}^{n+1,k+1} \in \mathbb{R}^{M_h}$  are the algebraic coefficient vectors of the velocity and pressure at the  $(k + 1)$ -th Picard iteration at time  $t^{n+1}$ . The matrix blocks are defined as follows:

- $F(\beta)$  is the velocity block. Before adding the stabilization terms, it is given by

$$F(\beta) = \frac{1}{\Delta t}M + \theta\nu A + \theta C(\beta). \quad (14)$$

Its components are:



---

– the velocity mass matrix

$$M_{ij} = \int_{\Omega} \phi_j \cdot \phi_i \, d\Omega; \quad (15)$$

– the velocity stiffness matrix

$$A_{ij} = \int_{\Omega} \nabla \phi_j : \nabla \phi_i \, d\Omega; \quad (16)$$

– the linearized convection matrix

$$C_{ij}(\boldsymbol{\beta}) = \int_{\Omega} ((\boldsymbol{\beta} \cdot \nabla) \phi_j) \cdot \phi_i \, d\Omega. \quad (17)$$

•  $B$  is the discrete divergence matrix:

$$B_{ij} = \int_{\Omega} \psi_i \nabla \cdot \phi_j \, d\Omega. \quad (18)$$

•  $-B^T$  is the discrete pressure-gradient block:

$$(-B^T)_{ij} = - \int_{\Omega} \psi_j \nabla \cdot \phi_i \, d\Omega. \quad (19)$$

The vector  $\mathbf{G}$  is obtained from the right-hand side functional of the fully-discrete formulation:

$$G_i = G(\phi_i). \quad (20)$$

The second block row has zero right-hand side because it represents the discrete incompressibility constraint. The choice of the convective velocity  $\boldsymbol{\beta}$  follows the Oseen or Picard strategy described above. The additional stabilization terms modify the algebraic system: Temam and grad-div affect the velocity block, while SUPG also contributes to the upper-right pressure-gradient block.

### 3.5 Stabilized Formulation

The previous block system represents the Galerkin discretization of the linearized problem. In convection-dominated regimes and in the presence of discrete incompressibility errors, additional stabilization terms are introduced to improve robustness. In the implementation we consider Temam, grad-div and SUPG stabilizations.

#### 3.5.1 Temam stabilization

The convective term is not exactly energy-preserving at the discrete level when the convective velocity  $\boldsymbol{\beta}$  is not exactly divergence-free. This may occur because the incompressibility constraint is only enforced in the finite-dimensional pressure space. To improve the energy behavior of the linearized convective operator, we use the Temam correction, defined by

$$t(\boldsymbol{\beta}; \mathbf{u}, \mathbf{v}) = \frac{1}{2} \int_{\Omega} (\nabla \cdot \boldsymbol{\beta}) \mathbf{u} \cdot \mathbf{v} \, d\Omega. \quad (21)$$

The convective form is therefore replaced by

$$c_{Temam}(\boldsymbol{\beta}; \mathbf{u}, \mathbf{v}) = c(\boldsymbol{\beta}, \mathbf{u}, \mathbf{v}) + t(\boldsymbol{\beta}; \mathbf{u}, \mathbf{v}). \quad (22)$$

If  $\boldsymbol{\beta}$  is exactly divergence-free, the correction vanishes and  $c_T$  reduces to the standard linearized convection form.

In the fully-discrete formulation, the implicit Temam contribution is added to the left-hand side as

$$\theta t(\boldsymbol{\beta}; \widehat{\mathbf{u}}_h^{n+1,k+1}, \mathbf{v}_h), \quad (23)$$

while the explicit contribution from the previous timestep enters the right-hand side with opposite sign:

$$-(1 - \theta) t(\mathbf{u}_h^n; \mathbf{u}_h^n, \mathbf{v}_h). \quad (24)$$

Thus, when the Temam stabilization is active, the velocity block is modified as

$$F_T(\boldsymbol{\beta}) = F(\boldsymbol{\beta}) + \theta T(\boldsymbol{\beta}), \quad (25)$$

where

$$T_{ij}(\boldsymbol{\beta}) = \frac{1}{2} \int_{\Omega} (\nabla \cdot \boldsymbol{\beta}) \phi_j \cdot \phi_i \, d\Omega. \quad (26)$$

The corresponding additional right-hand side term is

$$G_{(Temam),i} = -\frac{1 - \theta}{2} \int_{\Omega} (\nabla \cdot \mathbf{u}_h^n) \mathbf{u}_h^n \cdot \phi_i \, d\Omega. \quad (27)$$

This is the form implemented in the code: the Temam term depends on  $\boldsymbol{\beta}$  and is therefore assembled together with the timestep-dependent convection matrix.

### 3.5.2 Grad-div stabilization

The grad-div stabilization is introduced to improve the discrete enforcement of the incompressibility constraint. Although the finite element solution satisfies  $b(q_h, \mathbf{u}_h) = 0$  for all  $q_h \in Q_h$ , the pointwise divergence  $\nabla \cdot \mathbf{u}_h$  is not necessarily zero. This may affect both mass conservation and the quality of the pressure approximation. To penalize the divergence of the discrete velocity, we add the symmetric positive term

$$g(\mathbf{u}, \mathbf{v}) = \gamma \int_{\Omega} (\nabla \cdot \mathbf{u})(\nabla \cdot \mathbf{v}) \, d\Omega, \quad (28)$$

where  $\gamma \geq 0$  is the grad-div stabilization parameter.

At the fully-discrete level, the grad-div term is added to the momentum equation as

$$g(\widehat{\mathbf{u}}_h^{n+1,k+1}, \mathbf{v}_h). \quad (29)$$

Unlike the convective and Temam terms, this contribution does not depend on the linearization velocity  $\boldsymbol{\beta}$  and is therefore assembled in the time-independent part of the velocity block. The stabilized velocity block becomes

$$F_{GD}(\boldsymbol{\beta}) = F(\boldsymbol{\beta}) + G_{GD}, \quad (30)$$

where the grad-div matrix is defined by

$$(G_{GD})_{ij} = \gamma \int_{\Omega} (\nabla \cdot \phi_j)(\nabla \cdot \phi_i) \, d\Omega. \quad (31)$$

The effect of this term is to penalize velocity fields with large discrete divergence. For moderate values of  $\gamma$ , this improves mass conservation and can make the pressure field more robust. If  $\gamma$  is chosen too large, however, the velocity block may become more difficult to solve, increasing the cost of the linear solver. In the implementation, the parameter  $\gamma$  is controlled by the entry `Grad-div coefficient`.

### 3.5.3 SUPG stabilization

The Streamline-Upwind Petrov–Galerkin (SUPG) stabilization is introduced to improve robustness in convection-dominated regimes. The idea is to add a stabilizing contribution in the direction of the convective field  $\boldsymbol{\beta}$ , testing the momentum residual against the streamline direction

$$(\boldsymbol{\beta} \cdot \nabla) \mathbf{v}_h. \quad (32)$$

In the implementation, the SUPG contribution is built element by element. For each element  $K \in \mathcal{T}_h$ , we define a stabilization parameter

$$\tau_K = \left[ \left( \frac{2}{\Delta t} \right)^2 + \left( \frac{2\|\boldsymbol{\beta}\|}{h_K} \right)^2 + \left( \frac{4\nu}{h_K^2} \right)^2 \right]^{-1/2}, \quad (33)$$

where  $h_K$  is the element diameter and  $\|\boldsymbol{\beta}\|$  is the local magnitude of the convective velocity.

The SUPG term used in the code corresponds to the transient, convective and pressure-gradient part of the linearized momentum residual. On each element, it adds the following contribution:

$$\sum_{K \in \mathcal{T}_h} \tau_K \left( \frac{1}{\Delta t} \hat{\mathbf{u}}_h^{n+1,k+1} + (\boldsymbol{\beta} \cdot \nabla) \hat{\mathbf{u}}_h^{n+1,k+1} + \nabla \hat{p}_h^{n+1,k+1}, (\boldsymbol{\beta} \cdot \nabla) \mathbf{v}_h \right)_K. \quad (34)$$

The corresponding right-hand side contribution is

$$\sum_{K \in \mathcal{T}_h} \tau_K \left( \frac{1}{\Delta t} \mathbf{u}_h^n + \mathbf{f}^{n+1}, (\boldsymbol{\beta} \cdot \nabla) \mathbf{v}_h \right)_K. \quad (35)$$

At the algebraic level, the velocity-velocity SUPG contribution is

$$(S_{\text{SUPG}})_{ij} = \sum_{K \in \mathcal{T}_h} \tau_K \int_K \left[ \frac{1}{\Delta t} \phi_j + (\boldsymbol{\beta} \cdot \nabla) \phi_j \right] \cdot [(\boldsymbol{\beta} \cdot \nabla) \phi_i] dK. \quad (36)$$

Moreover, since the strong momentum residual contains the pressure gradient  $\nabla p$ , SUPG also adds a pressure-velocity coupling term in the momentum equation:

$$(P_{\text{SUPG}})_{ij} = \sum_{K \in \mathcal{T}_h} \tau_K \int_K \nabla \psi_j \cdot [(\boldsymbol{\beta} \cdot \nabla) \phi_i] dK. \quad (37)$$

Thus, when SUPG is active, the velocity block and the upper-right pressure block are modified by additional streamline terms. The right-hand side is also augmented by

$$(G_{\text{SUPG}})_i = \sum_{K \in \mathcal{T}_h} \tau_K \int_K \left( \frac{1}{\Delta t} \mathbf{u}_h^n + \mathbf{f}^{n+1} \right) \cdot [(\boldsymbol{\beta} \cdot \nabla) \phi_i] dK. \quad (38)$$

This formulation is consistent with the implemented code: the streamline test function is computed as  $(\boldsymbol{\beta} \cdot \nabla) \phi_i$ , the velocity residual contains the transient and convective parts, and the pressure contribution is included through  $\nabla \psi_j$ . Since the stabilization depends on  $\boldsymbol{\beta}$  and on  $\tau_K$ , it is assembled together with the timestep-dependent convection terms.

---

**Resulting stabilized algebraic system.** Combining the Galerkin discretization with the stabilization terms described above, the linear system solved at each timestep, and at each Picard iteration when the Picard strategy is active, can be written as

$$\begin{pmatrix} F_{\text{stab}}(\boldsymbol{\beta}) & -B^T + P_{\text{SUPG}}(\boldsymbol{\beta}) \\ B & 0 \end{pmatrix} \begin{pmatrix} \hat{\mathbf{U}}^{n+1,k+1} \\ \hat{\mathbf{P}}^{n+1,k+1} \end{pmatrix} = \begin{pmatrix} \mathbf{G}_{\text{stab}} \\ 0 \end{pmatrix}. \quad (39)$$

The stabilized velocity block is

$$F_{\text{stab}}(\boldsymbol{\beta}) = F(\boldsymbol{\beta}) + \theta T(\boldsymbol{\beta}) + G_{GD} + S_{\text{SUPG}}(\boldsymbol{\beta}), \quad (40)$$

where  $F(\boldsymbol{\beta})$  contains the mass, viscous and linearized convective contributions,  $T(\boldsymbol{\beta})$  is the Temam correction,  $G_{GD}$  is the grad-div matrix, and  $S_{\text{SUPG}}(\boldsymbol{\beta})$  is the velocity-velocity SUPG contribution.

The upper-right block is also modified by SUPG through the pressure-gradient streamline term  $P_{\text{SUPG}}(\boldsymbol{\beta})$ . Therefore, when SUPG is active, the pressure contribution in the momentum equation is not represented only by the standard block  $-B^T$ , but by the stabilized block

$$-B^T + P_{\text{SUPG}}(\boldsymbol{\beta}). \quad (41)$$

The right-hand side is consistently modified as

$$\mathbf{G}_{\text{stab}} = \mathbf{G} + \mathbf{G}_{\text{Temam}} + \mathbf{G}_{\text{SUPG}}, \quad (42)$$

where  $\mathbf{G}_{\text{Temam}}$  is the explicit Temam contribution from the previous timestep and  $\mathbf{G}_{\text{SUPG}}$  is the SUPG right-hand side term.

This final system preserves the saddle-point structure of the incompressible Navier–Stokes equations, but includes additional terms that improve the robustness of the discretization. Temam stabilization improves the energy behavior of the convective operator, grad-div stabilization penalizes violation of the incompressibility constraint, and SUPG adds streamline diffusion in convection-dominated regimes. The solution of this stabilized algebraic system, and the construction of efficient preconditioners for it, are discussed in the next section.

## 4 Finite Element Implementation

This section describes how the fully-discrete stabilized formulation introduced in the previous section is implemented. The goal is not to repeat the mathematical derivation, but to explain how the computational domain, finite element spaces, boundary conditions and matrix contributions are constructed in the code. Particular attention is paid to the separation between static operators, assembled once, and time-dependent operators, updated at each timestep and, when Picard iteration is used, at each nonlinear iteration.

### 4.1 Mesh Generation

The computational meshes are generated with Gmsh and then imported into the deal.II code through the physical tags associated with the boundary and fluid domain. Both the two-dimensional and three-dimensional configurations follow the DFG/Schaefer–Turek benchmark geometries.

For the two-dimensional flow past a cylinder, the computational domain is the rectangular channel

$$\Omega_{2D} = (0, 2.2) \times (0, 0.41),$$

with a circular obstacle of radius  $R = 0.05$  centered at

$$\mathbf{x}_c = (0.20, 0.20).$$

Therefore, the cylinder diameter is  $D = 0.1$ , which is used as the characteristic length in the definition of the Reynolds number. The geometry is shown in Figure 1.

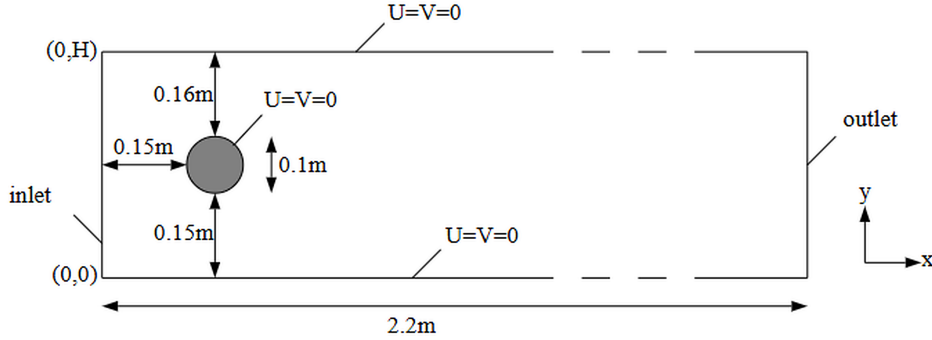


Figure 1: Two-dimensional DFG/Schaefer–Turek benchmark geometry adopted in this work. The channel has length 2.2 and height 0.41, while the cylinder has diameter  $D = 0.1$ .

For the three-dimensional case, the benchmark geometry is obtained by extending the channel in the spanwise direction. The computational domain is

$$\Omega_{3D} = (0, 2.5) \times (0, 0.41) \times (0, 0.41),$$

and the cylindrical obstacle has radius  $R = 0.05$ . Its axis is aligned with the  $z$ -direction and its center in the  $xy$ -plane is

$$(x_c, y_c) = (0.50, 0.20).$$

Thus the cylinder again has diameter  $D = 0.1$ . The distance between the inlet and the front of the cylinder is 0.45, while the distance between the rear of the cylinder and the outlet is 1.95. The three-dimensional geometry is shown in Figure 2.

In the two-dimensional case, triangular meshes are used, while in the three-dimensional case the fluid domain is discretized with tetrahedral elements. The mesh is refined near the cylinder boundary, where large velocity and pressure gradients occur, and in the wake region downstream of the obstacle, where vortex structures develop. A milder refinement is also imposed near the no-slip walls, while the far-field region is kept coarser in order to reduce the total number of degrees of freedom.

Figure 3 shows the level-0 two-dimensional mesh used to illustrate the refinement strategy. Higher mesh levels are obtained by reducing the characteristic mesh sizes in the Gmsh script and are used for more accurate benchmark computations.

The physical boundary labels are assigned in Gmsh and used in the code to impose the corresponding boundary conditions. In the two-dimensional case, the boundary is decomposed as

$$\Gamma_{\text{in}} : \text{inlet}, \quad \Gamma_{\text{out}} : \text{outlet}, \quad \Gamma_{\text{wall}} : \text{channel walls}, \quad \Gamma_{\text{cyl}} : \text{cylinder boundary}.$$

In the three-dimensional case, the same convention is used, with  $\Gamma_{\text{wall}}$  also including the front and back channel walls.

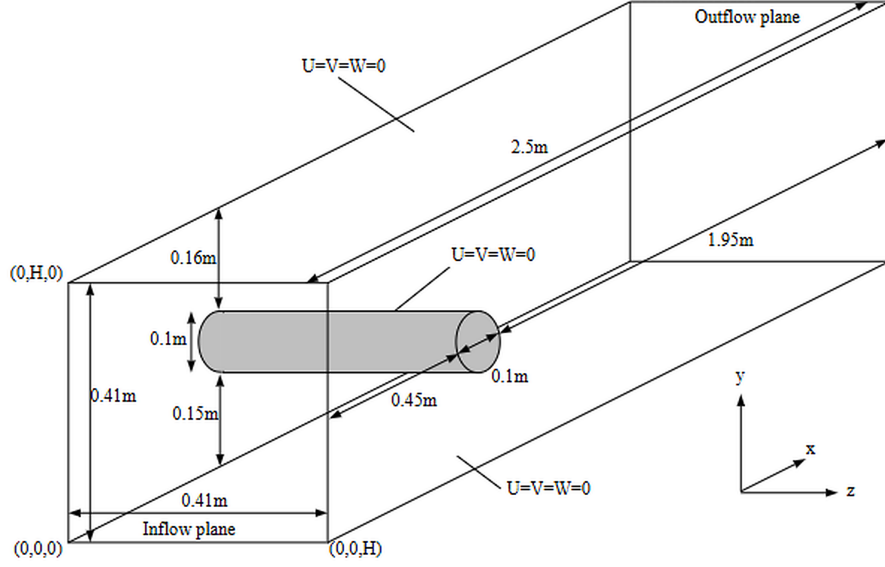


Figure 2: Three-dimensional DFG/Schaefer–Turek benchmark geometry adopted in this work. The cylinder axis is aligned with the  $z$ -direction and the channel has dimensions  $2.5 \times 0.41 \times 0.41$ .

## 4.2 Finite Element Spaces

In the implementation, we use the Taylor–Hood pair already discussed in Section 3.1.1, namely continuous quadratic elements for the velocity and continuous linear elements for the pressure:

$$\mathbf{u}_h \in [\mathbb{P}_2]^d, \quad p_h \in \mathbb{P}_1.$$

This pair is built using a vector-valued finite element for the velocity and a scalar finite element for the pressure. The global finite element space is therefore a mixed space containing  $d$  velocity components and one pressure component. The degrees of freedom are split into two blocks: one block for the velocity unknowns and one block for the pressure unknowns. This block decomposition is the one used in the algebraic saddle-point system introduced in Section 3.4.

The same implementation strategy is used in both two and three space dimensions. The only difference is the number of velocity components: two in the planar case and three in the three-dimensional case. Thus, the algebraic structure of the problem remains the same, while the size of the velocity block increases in the three-dimensional simulations.

## 4.3 Boundary Conditions

The boundary conditions are imposed through the physical boundary identifiers defined in the Gmsh mesh. In both the two-dimensional and three-dimensional configurations, the inlet, outlet, solid walls and cylinder boundary are assigned distinct physical tags. These tags are read by the code and used to associate the corresponding boundary functions with the finite element problem.

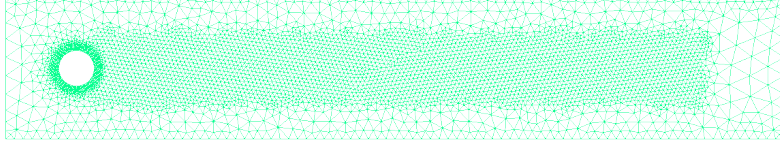


Figure 3: Level-0 two-dimensional mesh for the flow past a cylinder. The mesh is refined near the cylinder and in the wake region.

On the inlet boundary, a prescribed velocity profile is imposed as a Dirichlet condition. Two types of inlet profiles can be considered. The first one is a constant uniform inlet velocity, given in two dimensions by

$$\mathbf{u}_{\text{in}}(t) = (r(t)U_{\text{in}}, 0), \quad (43)$$

and in three dimensions by

$$\mathbf{u}_{\text{in}}(t) = (r(t)U_{\text{in}}, 0, 0). \quad (44)$$

Here  $U_{\text{in}}$  is the constant inlet speed and  $r(t)$  is an optional time ramp.

The second one is the parabolic profile imposed across the channel height. In the two-dimensional case:

$$\mathbf{u}_{\text{in}}(y, t) = \left( r(t) \frac{4U_m y(H - y)}{H^2}, 0 \right), \quad (45)$$

where  $H = 0.41$  is the channel height and  $U_m$  is the maximum inlet velocity. This profile vanishes at the top and bottom walls and reaches its maximum value at the center of the inlet section.

In the three-dimensional case, the inlet profile is defined on the rectangular cross-section:

$$\mathbf{u}_{\text{in}}(y, z, t) = \left( r(t) \frac{16U_m y(H - y)z(W - z)}{H^2 W^2}, 0, 0 \right), \quad (46)$$

where  $H = 0.41$  is the channel height and  $W = 0.41$  is the channel width. This profile vanishes on the whole boundary of the inlet cross-section and reaches its maximum value in the interior.

The time factor  $r(t)$  is an optional time-dependent modulation applied to the inlet velocity in the ramp benchmark. In the implementation it is defined as:

$$r(t) = \sin \left( \frac{\pi t}{T_{\text{ramp}}} \right) \quad (47)$$

If  $r(t) = 1$  no ramp is used.

On the channel walls and on the cylinder boundary, homogeneous Dirichlet conditions are imposed:

$$\mathbf{u} = \mathbf{0} \quad \text{on } \Gamma_{\text{wall}} \cup \Gamma_{\text{cyl}}. \quad (48)$$

These conditions model the no-slip constraint on the solid boundaries.

At the outlet, a pressure condition is imposed in weak form as a natural boundary condition. In the implementation, the prescribed outlet pressure  $p_{\text{out}}$  contributes to the right-hand side through

$$- \int_{\Gamma_{\text{out}}} p_{\text{out}} \mathbf{n} \cdot \mathbf{v}_h d\sigma. \quad (49)$$

In the benchmark simulations considered here,  $p_{\text{out}} = 0$ . Therefore, the outlet acts as a do-nothing outflow condition for the velocity, while fixing the pressure level through the prescribed outlet pressure.

The same boundary-condition structure is used in both two and three space dimensions. The main difference is that, in three dimensions,  $\Gamma_{\text{wall}}$  also includes the front and back walls of the channel, in addition to the top and bottom walls.

## 4.4 Assembly of the System

The global linear system is assembled by separating the contributions that do not depend on the convective field from those that must be updated at each timestep. This separation follows the implementation in the code: the static matrix is built once, while the time-dependent matrix and the right-hand side are rebuilt whenever the convective velocity  $\boldsymbol{\beta}$  changes.

At each timestep, the final system matrix is obtained as

$$A_{\text{sys}}(\boldsymbol{\beta}) = A_{\text{static}} + A_{\text{time}}(\boldsymbol{\beta}). \quad (50)$$

In the Oseen case, this update is performed once per timestep, with  $\boldsymbol{\beta} = \mathbf{u}_h^n$ . In the Picard case, it is performed at each nonlinear iteration, with  $\boldsymbol{\beta} = \mathbf{u}_h^{n+1,k}$ .

### 4.4.1 Assembly of static matrices

The static assembly contains all terms that are independent of the current convective velocity. In particular, it includes the velocity mass matrix, the viscous stiffness matrix, the pressure-velocity coupling blocks, and the grad-div stabilization term. At the algebraic level, the static matrix has the form

$$A_{\text{static}} = \begin{pmatrix} \frac{1}{\Delta t} M_u + \theta \nu A_u + G_{GD} & -B^T \\ B & 0 \end{pmatrix}. \quad (51)$$

The velocity mass matrix is defined by

$$(M_u)_{ij} = \int_{\Omega} \phi_j \cdot \phi_i d\Omega, \quad (52)$$

and enters the monolithic system through the term  $\Delta t^{-1} M_u$ . The viscous stiffness matrix is

$$(A_u)_{ij} = \int_{\Omega} \nabla \phi_j : \nabla \phi_i d\Omega, \quad (53)$$



and is multiplied by  $\theta\nu$  in the static block.

The incompressibility constraint and pressure-gradient contribution are assembled through the matrices

$$B_{ij} = \int_{\Omega} \psi_i \nabla \cdot \phi_j d\Omega, \quad (54)$$

and

$$(-B^T)_{ij} = - \int_{\Omega} \psi_j \nabla \cdot \phi_i d\Omega. \quad (55)$$

These two blocks define the saddle-point structure of the discrete Navier–Stokes problem.

When grad-div stabilization is active, the additional matrix

$$(G_{GD})_{ij} = \gamma \int_{\Omega} (\nabla \cdot \phi_j)(\nabla \cdot \phi_i) d\Omega \quad (56)$$

is added to the velocity block. Since this term depends only on the basis functions and on the parameter  $\gamma$ , it is assembled together with the static contributions.

In addition to the monolithic matrix, the implementation may assemble auxiliary matrices required by the preconditioners. In particular, the pure velocity mass matrix  $M_u$  and the pressure mass matrix

$$(M_p)_{ij} = \int_{\Omega} \psi_j \psi_i d\Omega \quad (57)$$

are constructed only when required by the selected preconditioner. This avoids building unnecessary matrices when they are not used.

#### 4.4.2 Assembly of time-dependent matrices

The time-dependent assembly contains the terms that depend on the current convective field  $\beta$ , on the previous solution  $\mathbf{u}_h^n$ , and on the current time level. These contributions are stored separately and then added to the static matrix.

The main time-dependent contribution is the Oseen/Picard convection matrix

$$C_{ij}(\beta) = \int_{\Omega} ((\beta \cdot \nabla) \phi_j) \cdot \phi_i d\Omega, \quad (58)$$

which enters the system as  $\theta C(\beta)$ . If Temam stabilization is enabled, the correction defined in Section 3.5.1 is assembled at this stage, since it depends on the current convective field  $\beta$ .

When SUPG stabilization is active, the implementation also assembles the streamline terms associated with the transient, convective and pressure-gradient parts of the residual. The velocity-velocity SUPG contribution is added to the time-dependent matrix, while the pressure-gradient SUPG contribution modifies the upper-right block of the monolithic system.

The right-hand side is assembled at each timestep as well. It contains the previous solution contribution

$$\frac{1}{\Delta t}(\mathbf{u}_h^n, \phi_i), \quad (59)$$

the force term evaluated according to the  $\theta$ -method, and the explicit parts of diffusion and convection:

$$\begin{aligned} G_i = & \frac{1}{\Delta t}(\mathbf{u}_h^n, \phi_i) + (\theta \mathbf{f}^{n+1} + (1 - \theta) \mathbf{f}^n, \phi_i) \\ & - (1 - \theta) \nu (\nabla \mathbf{u}_h^n, \nabla \phi_i) - (1 - \theta) ((\mathbf{u}_h^n \cdot \nabla) \mathbf{u}_h^n, \phi_i). \end{aligned} \quad (60)$$

If Temam and SUPG stabilizations are active, their corresponding explicit right-hand side contributions are added consistently.

The outlet pressure condition is also assembled during the time-dependent stage. For a prescribed outlet pressure  $p_{\text{out}}$ , the contribution is

$$- \int_{\Gamma_{\text{out}}} p_{\text{out}} \mathbf{n} \cdot \boldsymbol{\phi}_i d\sigma. \quad (61)$$

Before evaluating boundary data, the time-dependent boundary functions are updated to the current time.

Finally, after the static and time-dependent contributions have been combined, Dirichlet boundary conditions are imposed strongly on the velocity components. Only velocity degrees of freedom are constrained, while pressure degrees of freedom remain unconstrained. This produces the final linear system solved by the Krylov method.

## 4.5 Force coefficients and pressure probes

The drag and lift coefficients are computed by evaluating the force exerted by the fluid on the cylindrical obstacle. Let  $\Gamma_{\text{cyl}}$  denote the cylinder boundary and let  $\mathbf{n}$  be the unit normal vector pointing outward from the fluid domain. The Cauchy stress tensor used for the force evaluation is

$$\boldsymbol{\sigma}(\mathbf{u}, p) = \nu (\nabla \mathbf{u} + \nabla \mathbf{u}^T) - pI.$$

Since  $\mathbf{n}$  is the outward normal of the fluid domain, the force exerted by the fluid on the obstacle is obtained with the opposite sign:

$$\mathbf{F} = - \int_{\Gamma_{\text{cyl}}} \boldsymbol{\sigma}(\mathbf{u}_h, p_h) \mathbf{n} d\gamma.$$

In two dimensions, the drag and lift forces are the components of  $\mathbf{F}$  in the streamwise and transverse directions:

$$F_D = \mathbf{F} \cdot \mathbf{e}_x, \quad F_L = \mathbf{F} \cdot \mathbf{e}_y.$$

The corresponding dimensionless coefficients are normalized using the reference velocity  $U_{\text{ref}}$ , the cylinder diameter  $D$ , and unit density:

$$C_D = \frac{F_D}{\frac{1}{2} U_{\text{ref}}^2 D}, \quad C_L = \frac{F_L}{\frac{1}{2} U_{\text{ref}}^2 D}.$$

In the three-dimensional case, the same definition is used, but the reference length is replaced by the projected reference area of the cylinder,

$$A_{\text{ref}} = D L_z,$$

where  $L_z$  is the spanwise length of the cylinder. Therefore,

$$C_D = \frac{F_D}{\frac{1}{2} U_{\text{ref}}^2 A_{\text{ref}}}, \quad C_L = \frac{F_L}{\frac{1}{2} U_{\text{ref}}^2 A_{\text{ref}}}.$$

---

The pressure drop used in the benchmark validation is computed as the difference between the pressure evaluated at two prescribed probe points located upstream and downstream of the cylinder:

$$\Delta p = p(\mathbf{x}_{\text{up}}) - p(\mathbf{x}_{\text{down}}).$$

This quantity, together with the drag and lift coefficients, provides the main physical output used to compare the numerical solution with the reference benchmark data. In the current implementation, the pressure at each probe is approximated by the pressure value at the closest boundary quadrature point to the prescribed probe location. Therefore, the reported pressure drop is a nearest-sample approximation of the benchmark pressure drop, rather than an exact finite-element point evaluation.

## 5 Preconditioners

### 5.1 Block factorization and Schur complement

The space-time discretisation of the incompressible Navier–Stokes equations yields, at each time step, a large, sparse, and indefinite linear system of saddle-point type. Systems of this form are typically badly-conditioned and non-symmetric; standard Krylov subspace solvers such as GMRES, Generalized Minimal Residuals, therefore exhibit prohibitively slow convergence in the absence of an effective preconditioner. The natural strategy for exploiting the block structure of  $\mathcal{A}$  is to construct a *Block Preconditioner* that approximates the inverse of each sub-block separately, combining them through a triangular factorisation. The exact block LU factorisation of  $\mathcal{A}$  reads

$$\mathcal{A} = \underbrace{\begin{pmatrix} F & 0 \\ B & S \end{pmatrix}}_L \underbrace{\begin{pmatrix} I & -F^{-1}B^T \\ 0 & I \end{pmatrix}}_U, \quad (62)$$

where  $S = BF^{-1}B^T$  is the *Schur complement*. If both factors were applied exactly, the preconditioned operator would have a spectrum concentrated at a single value and GMRES would converge in at most two iterations [3]. In practice, neither  $F^{-1}$  and  $S^{-1}$  are available in closed form; the four preconditioners described below differ precisely in how they construct tractable approximations to these operators. This gives the possibility to approximate the inverse of each block separately and combine them through a suitable triangle factorization.

Throughout this section we adopt the following general form for the preconditioner:

$$P = \begin{pmatrix} F & 0 \\ B & -\tilde{S} \end{pmatrix} \begin{pmatrix} I & \tilde{F}^{-1}B^T \\ 0 & I \end{pmatrix}. \quad (63)$$

The preconditioner is applied from the right:  $\mathcal{A}P^{-1}\mathbf{y} = \mathbf{b}$ ,  $\mathbf{x} = P^{-1}\mathbf{y}$ , at each FGMRES iteration the current residual  $\mathbf{r} = (\mathbf{r}_u, \mathbf{r}_p)^T$  is transformed into the correction  $\mathbf{z} = P^{-1}\mathbf{r} = (\mathbf{z}_u, \mathbf{z}_p)^T$ .

The solver implements four block preconditioning strategies: **SIMPLE**, **Block Triangular**, **Yosida**, and **PCD**. Among them, SIMPLE, Yosida and PCD follow the preconditioning framework discussed in [2], whereas the Block Triangular preconditioner is included as a standard Schur-complement based block preconditioner. All these methods are designed to exploit the saddle-point structure of the incompressible Navier–Stokes system [1] and to improve the robustness of the Krylov solver for large-scale distributed-memory computations. Inner linear systems are solved approximately using ILU-preconditioned GMRES, and all matrix operations rely on the `TrilinosWrappers` layer of `deal.II`.

### 5.2 SIMPLE Preconditioner

The Semi-Implicit Method for Pressure-Linked Equations (SIMPLE) Preconditioner is defined by the following block factorization:

$$P_{SIMPLE} = \begin{pmatrix} F & 0 \\ B & -\tilde{S}_{SIMPLE} \end{pmatrix} \begin{pmatrix} I & D^{-1}B^T \\ 0 & \alpha I \end{pmatrix}. \quad (64)$$

where  $D = \text{diag}(F)$  is the diagonal of the velocity block and  $\alpha \in (0, 1]$  is a *pressure relaxation parameter*. The key approximation consists in considering  $F^{-1} \approx D^{-1}$  in the

---

Schur complement, obtaining:  $\tilde{S}_{\text{SIMPLE}} = B D^{-1} B^T$ .

Additionally, a small pressure-mass shift is added to improve robustness of the pressure solve, hence:  $\tilde{S}_{\text{SIMPLE}} = B D^{-1} B^T + \varepsilon M_p$  with  $\varepsilon = 10^{-8}$ .

Since  $D^{-1}$  is a diagonal matrix,  $\tilde{S}_{\text{SIMPLE}}$  is assembled explicitly via a single sparse matrix-matrix product. The implementation proceeds through the following steps.

1. **Intermediate velocity solve.** Solve  $F \tilde{\mathbf{u}} = \mathbf{r}_u$  by means of an inner GMRES iteration with ILU preconditioner.
2. **Pressure residual update.** Forming  $\mathbf{r}_p^* = \mathbf{r}_p - B \tilde{\mathbf{u}}$
3. **Schur complement solve.** Solve  $\tilde{S}_{\text{SIMPLE}} \mathbf{z}_p = \mathbf{r}_p^*$  via a second inner GMRES with ILU preconditioner.
4. **Pressure relaxation.**  $\mathbf{z}_p = \alpha \mathbf{z}_p$
5. **Velocity correction.**  $\mathbf{z}_u = \tilde{\mathbf{u}} + D^{-1} B^T \mathbf{z}_p$

The quality of the SIMPLE approximation is governed by how well  $D$  represents  $F$ , this happens when the time step  $\Delta t$  is small, the mass term dominates  $F$ ; in that case the approximation is tight and convergence is rapid. For large time steps or high Reynolds numbers, however, the diagonal approximation deteriorates, and the outer iteration count grows. The relaxation parameter  $\alpha$  can be tuned to improve robustness.

### 5.3 Block Triangular Preconditioner

The Block Triangular preconditioner is based on the saddle-point structure of the linearized Navier-Stokes system

$$A = \begin{pmatrix} F & -B^T \\ B & 0 \end{pmatrix}.$$

With this sign convention, the pressure Schur complement is

$$S = B F^{-1} B^T.$$

An exact block triangular preconditioner would require the exact application of both  $F^{-1}$  and  $S^{-1}$ , which is not feasible for large-scale finite element problems. Therefore, the implemented preconditioner replaces  $F^{-1}$  in the Schur complement by the inverse of the diagonal of the velocity block:

$$D = \text{diag}(F), \quad \tilde{S}_{\text{BT}} = B D^{-1} B^T.$$

In the implementation, a small pressure-mass regularization is also added to improve the robustness of the pressure solve:

$$\tilde{S}_{\text{BT}} = B D^{-1} B^T + \varepsilon M_p, \quad \varepsilon = 10^{-8}.$$

The corresponding block triangular preconditioner can be written as

$$P_{\text{BT}} = \begin{pmatrix} F & -B^T \\ 0 & \tilde{S}_{\text{BT}} \end{pmatrix}.$$

The application of  $P_{\text{BT}}$  involves the following sequence of operations:

- 
1. **Schur complement solve.** Compute the pressure component  $z_p$  by approximately solving  $\tilde{S}_{\text{BT}} z_p = y_p$ , using an inner GMRES iteration preconditioned by ILU.
  2. **Velocity right-hand side correction.** Form the corrected velocity right-hand side  $y_u^* = y_u + B^T z_p$ . The plus sign is consistent with the fact that the monolithic system contains the pressure-gradient block  $-B^T$ .
  3. **Velocity solve.** Compute the velocity component  $z_u$  by approximately solving  $F z_u = y_u^*$ , again using an inner GMRES iteration preconditioned by ILU.

This preconditioner preserves the upper block triangular structure associated with the Schur-complement factorization of the saddle-point system. Its effectiveness depends on the quality of the diagonal approximation  $D^{-1} \simeq F^{-1}$ , which is expected to be more accurate when the mass contribution dominates the velocity block, for instance for small time steps.

## 5.4 Yosida Preconditioner

The Yosida preconditioner structurally resembles the SIMPLE preconditioner but differs in its formulation of the approximate Schur complement and the velocity correction step. The momentum inverse is approximated by  $\Delta t M_{u,c}^{-1}$ , where  $M_{u,c} = \text{diag}(M_u)$  denotes the diagonal approximation of the velocity mass matrix. The resulting Schur approximation is given by

$\tilde{S}_Y = B M_{u,c}^{-1} B^T$ , which represents a positive semidefinite approximation of the pressure Schur complement and it is solved with ILU preconditioning of the Schur approximation. The application of  $P_Y^{-1}$  involves the following sequence of operations:

1. **Intermediate velocity solve.**  $F \tilde{\mathbf{u}} = \mathbf{r}_u$
2. **Pressure residual and Schur solve.**  $\mathbf{r}_p^* = \mathbf{r}_p - B \tilde{\mathbf{u}}$  and  $\tilde{S}_Y \mathbf{z}_p = \mathbf{r}_p^*$
3. **Velocity correction via second F-solve.** Yosida performs a second approximate F-solve to correct the velocity  $F \mathbf{u}^* = -B^T \mathbf{z}_p$  and  $\mathbf{z}_u = \tilde{\mathbf{u}} - \mathbf{u}^*$ .

The additional inner solve in step 3, increases the cost per outer FGMRES iteration but yields a preconditioned operator that more faithfully approximates  $\mathcal{A}^{-1}$ , leading to a reduction in the total number of outer iterations, particularly for smaller time steps where the mass-matrix approximation of  $F^{-1}$  is most accurate [2].

## 5.5 PCD Preconditioner

The Pressure Convection–Diffusion (PCD) preconditioner [3] follows a fundamentally different philosophy: rather than assembling an explicit sparse Schur approximation, it approximates the action of  $S^{-1}$  entirely within *pressure space* through a sequence of pressure-space operators.

The starting observation is that the velocity-block operator  $F$  acts on velocity fields in a manner analogous to the pressure-space operator

$$F_p = \frac{1}{\Delta t} M_p + \theta \nu A_p + \theta C_p(\beta), \quad (65)$$

where  $M_p$ ,  $A_p$ , and  $C_p$  are the pressure mass, Laplacian, and convection matrices, respectively, and  $\boldsymbol{\beta}$  is the linearized convective velocity. When Temam stabilization is enabled, the corresponding scalar pressure-space correction is also added to  $F_p$ . Exploiting the approximate commutation relation  $BF^{-1} \approx F_p^{-1}B$ , one obtains

$$S = BF^{-1}B^T \approx F_p M_p^{-1} A_p \implies S^{-1} \approx \tilde{S}_{\text{PCD}}^{-1} = A_p^{-1} F_p M_p^{-1}. \quad (66)$$

The pressure Laplacian is assembled as the discrete weighted operator

$$A_p^{\text{disc}} = B \text{diag}(M_u)^{-1} B^T, \quad (67)$$

which provides a better spectral match to the Schur complement. Inflow boundary conditions on  $F_p$  are imposed via the Robin-type condition

$$\nu \nabla p \cdot \mathbf{n} + (\mathbf{u} \cdot \mathbf{n}) p = 0 \quad \text{on } \Gamma_{\text{in}}, \quad (68)$$

following the Elman–Tuminaro strategy [3], enforced weakly by adding the boundary integral  $\theta(\boldsymbol{\beta} \cdot \mathbf{n} \psi_j, \psi_i)_{\Gamma_{\text{in}}}$  during assembly, cell by cell in `assemble_timestep`. The PCD preconditioner is also a block triangular operator:

$$P_{\text{PCD}}^{-1} = \begin{pmatrix} F^{-1} & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} I & 0 \\ B & \tilde{S}_{\text{PCD}} \end{pmatrix}^{-1}, \quad (69)$$

where, like in BlockTriangular or in SIMPLE, a small pressure-mass regularization is added to improve the robustness of the pressure solve. Its application to  $(\mathbf{r}_u, \mathbf{r}_p)^T$  proceeds as follows:

1. **Approximate Schur solve (pressure block):** Compute  $\mathbf{z}_p \approx \tilde{S}_{\text{PCD}}^{-1} \mathbf{r}_p$  in three sub-steps:
  - (a)  $\mathbf{z}_1 \approx M_p^{-1} \mathbf{r}_p$  (single ILU application);
  - (b)  $\mathbf{z}_2 = F_p \mathbf{z}_1$  (sparse matrix–vector product);
  - (c)  $\mathbf{z}_p \approx A_p^{-1} \mathbf{z}_2$  (inner GMRES solve with ILU preconditioner).
2. **Velocity residual correction:**  $\mathbf{r}_u^* = \mathbf{r}_u - B^T \mathbf{z}_p$ .
3. **Velocity solve:**  $F \mathbf{z}_u \approx \mathbf{r}_u^*$  (inner GMRES solve with ILU preconditioner).

A key advantage of PCD is that it never assembles the Schur complement explicitly; the entire pressure correction is computed through two pressure-space solves and one matrix–vector product. Consequently, the method is less sensitive to mesh refinement and Reynolds number than SIMPLE or Yosida, at the cost of a more involved setup phase that requires the assembly of  $F_p$  and  $A_p^{\text{disc}}$  alongside the standard system matrices.

From a performance standpoint, all preconditioners are implemented in a distributed-memory setting through the `TrilinosWrappers` layer of `deal.II`. SIMPLE, Yosida and PCD follow the block preconditioning strategies discussed in [2], while the Block Triangular preconditioner is included as a standard Schur-complement based block preconditioner.

SIMPLE, Block Triangular and Yosida require the construction of an explicit sparse approximation of the pressure Schur complement, obtained by replacing the inverse of the

velocity block with a diagonal approximation. In particular, SIMPLE and Block Triangular use  $D = \text{diag}(F)$ , whereas Yosida uses a diagonal approximation of the velocity mass matrix. The assembly of these Schur approximations involves sparse matrix operations with the divergence block  $B$ , whose cost may grow on fine meshes and may introduce additional communication overhead in parallel runs.

The PCD preconditioner follows a different approach. Instead of approximating the Navier–Stokes Schur complement directly through  $BD^{-1}B^T$ , it approximates the action of the inverse Schur complement by combining pressure-space operators, namely the pressure mass matrix, the pressure convection–diffusion operator and the pressure Laplacian. This makes PCD more closely related to the differential structure of the Schur complement and can improve robustness with respect to mesh refinement and Reynolds number, at the price of a more involved setup phase.

In all cases, the outer monolithic system is solved by right-preconditioned FGMRES, while the inner systems arising in the application of the preconditioners are solved only approximately by GMRES with ILU preconditioning. The numerical comparison in the following section therefore focuses on both convergence behaviour, measured through the outer FGMRES iterations, and computational cost, including setup and preconditioner application time.

## 6 Results and Validation

The physical validation benchmarks are denoted by  $V_i$  and are compared with the Schaefer–Turek reference data whenever a direct reference interval is available [4]. The preconditioner benchmarks are denoted by  $P_i$  and are discussed in the same spirit as the preconditioner study of Deparis–Grandperrin–Quarteroni [2]: the relevant quantities are the outer FGMRES iteration count, the setup and linear solve costs, the dependence on  $\Delta t$ , viscosity, mesh size, and the strong-scaling behaviour.

Throughout the benchmark descriptions,  $U_{\text{in}}$  denotes the peak inlet velocity that scales the imposed profile, not the cross-section average. In the two-dimensional channel the inlet boundary condition is the parabolic profile

$$u_x(y, t) = f(t) \frac{4U_{\text{in}}y(H - y)}{H^2}, \quad u_y = 0,$$

so the centerline velocity is  $U_{\text{in}}f(t)$  and the section-mean inlet velocity is  $2U_{\text{in}}f(t)/3$ . Steady and periodic cases use  $f(t) = 1$ . Ramped cases use the Schaefer–Turek half-sine time profile  $f(t) = \sin(\pi t/T_{\text{ramp}})$ , with  $T_{\text{ramp}} = 8$ , so the inlet is switched on and then back to zero over the benchmark window. The three-dimensional case uses the analogous tensor-product parabolic profile over the rectangular inlet section.

### 6.1 Physical Validation Benchmarks

The validation campaign uses the Schaefer–Turek flow-past-cylinder geometry: a two-dimensional channel of height  $H = 0.41$  with a cylinder of diameter  $D = 0.1$ . The spatial discretization is Taylor–Hood  $P_2/P_1$ , and the time integration uses the  $\theta$ -method with  $\theta = 0.6$ . Unless otherwise stated, the default mesh is L0, the time step is  $\Delta t = 10^{-3}$ , Temam, grad-div and SUPG stabilization are enabled. The preconditioner used is PCD.



The force coefficients and pressure drop are computed according to the definitions given in Section 4.5. In particular, the Schaefer–Turek pressure-drop convention is

$$\Delta p = p(a_2) - p(a_1),$$

where  $a_2$  is the upstream probe and  $a_1$  is the downstream probe.

For periodic shedding cases, the vortex-shedding frequency is reported through the Strouhal number,

$$\text{St} = \frac{D}{U_{\text{ref}} \bar{T}},$$

where  $\bar{T}$  is the mean shedding period measured from consecutive positive lift peaks after the initial transient.

For each quantity with a Schaefer–Turek reference interval, the error column reports the signed distance to the nearest violated bound:  $e_{\partial I} = 0$  inside the interval,  $e_{\partial I} = x - x_{\min}$  below the lower bound, and  $e_{\partial I} = x - x_{\max}$  above the upper bound.

Table 1: Overview of the physical validation benchmarks.

| Benchmark | Case                     | Purpose                                                                                                           |
|-----------|--------------------------|-------------------------------------------------------------------------------------------------------------------|
| V1        | 2D-1 steady validation   | Low-Reynolds-number steady cylinder case; compares final drag, lift, pressure drop and divergence.                |
| V2        | 2D-2 periodic validation | Periodic shedding case; validates the Strouhal number and compares drag/lift extrema and pressure drop.           |
| V3        | 2D-3 ramp validation     | Baseline ramp case on the coarse mesh; checks drag peak, lift peak and final pressure drop.                       |
| V4        | Mesh refinement 2D-3     | Repeats the ramp case on L0 and L1 to assess spatial refinement and validation quality.                           |
| V5        | Timestep refinement 2D-3 | Sweeps $\Delta t$ on L0 to measure temporal-discretization effects on force extrema and pressure drop.            |
| V6        | Reynolds sweep 2D-3      | Diagnostic $Re$ sweep at $Re = 20$ , $Re = 100$ , and $Re = 200$ to assess viscosity sensitivity and lift growth. |
| V7        | Grad-div stabilization   | Sweeps $\gamma$ to quantify incompressibility improvement and its cost/iteration impact.                          |
| V8        | 3D validation            | Coarse 3D-3Z validation run; compares 3D drag, lift and pressure-drop diagnostics.                                |

### 6.1.1 V1 – 2D-1 Steady Validation

Setup:  $U_{\text{in}} = 0.3$  and  $Re = 20$ . Benchmark V1 corresponds to the steady low-Reynolds-number Schaefer–Turek case 2D-1. The final pressure drop is very close to the reference interval, while the drag is slightly below the lower bound and the lift is slightly above the reference interval. The quantitative comparison is reported in Table 2, while Figures 4 and 5 show the pressure and velocity fields.

Table 2: V1 comparison with the Schaefer–Turek 2D-1 reference interval.

| Quantity                   | Present result | Reference interval | $e_{\partial I}$ |
|----------------------------|----------------|--------------------|------------------|
| $C_D(T)$                   | 5.53875        | 5.5700–5.5900      | -0.03125         |
| $C_L(T)$                   | 0.01230        | 0.0104–0.0110      | +0.00130         |
| $\Delta p(T)$              | 0.11711        | 0.1172–0.1176      | -0.00009         |
| $\ \nabla \cdot u\ _{L^2}$ | 0.00567        | –                  | –                |



Figure 4: Benchmark V1: pressure field at  $Re = 20$  and  $t = 4$  s.

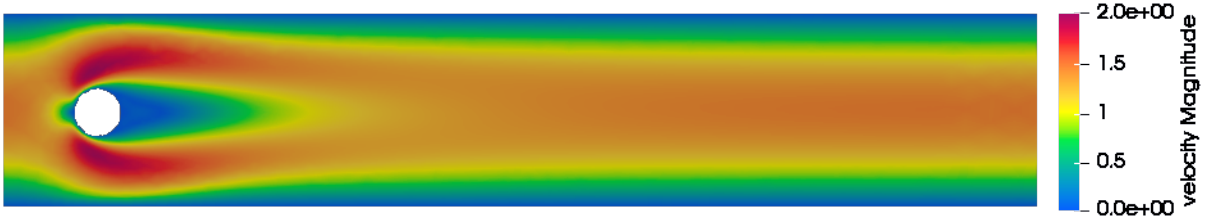


Figure 5: Benchmark V1: velocity magnitude at  $Re = 20$  and  $t = 4$  s.

### 6.1.2 V2 – 2D-2 Periodic Validation

Setup:  $U_{\text{in}} = 1.5$  and  $Re = 100$ . Benchmark V2 corresponds to the periodic  $Re = 100$  Schaefer–Turek case 2D-2. The reported statistics are computed after the transient, over the interval  $t \geq 10$ . The Strouhal number is inside the reference range, indeed  $\bar{T} = 0.33331$  and  $St = 0.30002$ . The drag peak is slightly below the reference interval, while the lift peak is slightly above it. The pressure drop reported for the reference comparison is evaluated at  $t_0 + (t_1 - t_0)/2 = 19.811$ , as prescribed by the benchmark, and lies inside the reference interval. The complete reference comparison is summarized in Table 3.

Table 3: V2 comparison with the Schaefer–Turek 2D-2 reference interval.

| Quantity                    | Present result | Reference interval | $e_{\partial I}$ |
|-----------------------------|----------------|--------------------|------------------|
| $C_D^{\max}$                | 3.20867        | 3.2200–3.2400      | -0.01133         |
| $C_L^{\max}$                | 1.03803        | 0.9900–1.0100      | +0.02803         |
| $St$                        | 0.30002        | 0.2950–0.3050      | 0                |
| $\Delta p(t_0 + \bar{T}/2)$ | 2.49782        | 2.4600–2.5000      | 0                |

The direct validation statement for V2 is therefore mixed: the shedding frequency is validated, but the force amplitudes still show a few percent discrepancy on the available mesh.

### 6.1.3 V3 – 2D-3 Ramp Validation on L0

Setup:  $U_{\text{in}} = 1.5$  and  $Re = 100$ . Benchmark V3 is the basic 2D-3 ramp validation on mesh L0. The pressure drop at  $t = 8$  lies inside the Schaefer–Turek reference interval. The maximum drag is below the lower bound and the maximum lift is above the upper bound, indicating that the coarse L0 mesh captures the transient qualitatively but is not completely sufficient for the final quantitative validation. The quantitative comparison is reported in Table 4, while Figures 6 and 7 show the corresponding pressure and velocity fields.

Table 4: V3 comparison with the Schaefer–Turek 2D-3 reference interval.

| Quantity                   | Present result | Reference interval   | $e_{\partial I}$ |
|----------------------------|----------------|----------------------|------------------|
| $C_D^{\max}$               | 2.91890        | 2.9300–2.9700        | -0.01110         |
| $C_L^{\max}$               | 0.51283        | 0.4700–0.4900        | +0.02283         |
| $\Delta p(8)$              | -0.11151       | $[-0.1150, -0.1050]$ | 0                |
| $\ \nabla \cdot u\ _{L^2}$ | 0.04274        | –                    | –                |



Figure 6: Benchmark V3: pressure field at  $Re = 100$  and  $t = 4$  s.

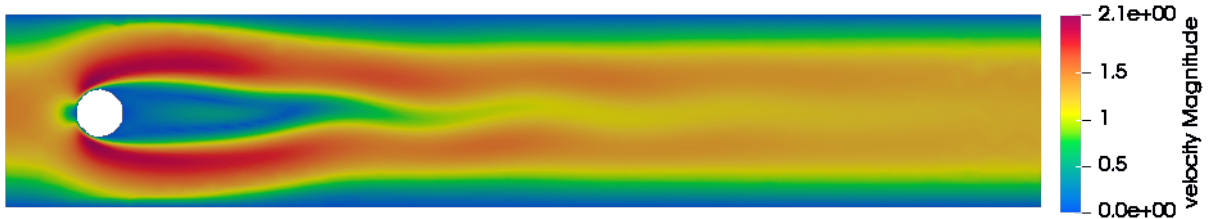


Figure 7: Benchmark V3: velocity magnitude at  $Re = 100$  and  $t = 4$  s.

### 6.1.4 V4 – Mesh Refinement on 2D-3

Setup:  $U_{\text{in}} = 1.5$  and  $Re = 100$  for both meshes. Benchmark V4 repeats the 2D-3 ramp case on L0 and L1. The L1 mesh has 163,231 degrees of freedom, compared with 40,108 on L0. Spatial refinement improves the physical validation substantially:  $C_D^{\max}$  moves inside the Schaefer–Turek interval and the pressure drop remains validated. The lift peak is still slightly above the upper reference bound, but the discrepancy is small. The mesh-refinement comparison is reported in Table 5, and the corresponding signed errors are given in Table 6.

Table 5: V4 mesh-refinement results for the 2D-3 ramp case.

| Mesh      | DoFs    | $C_D^{\max}$  | $C_L^{\max}$  | $\Delta p(8)$      |
|-----------|---------|---------------|---------------|--------------------|
| L0        | 40,108  | 2.91890       | 0.51283       | -0.11151           |
| L1        | 163,231 | 2.93839       | 0.49118       | -0.11121           |
| Reference | –       | 2.9300–2.9700 | 0.4700–0.4900 | [−0.1150, −0.1050] |

Table 6: V4 signed error to the nearest Schaefer–Turek bound.

| Mesh | $e_{\partial I}(C_D^{\max})$ | $e_{\partial I}(C_L^{\max})$ | $e_{\partial I}(\Delta p(8))$ |
|------|------------------------------|------------------------------|-------------------------------|
| L0   | -0.01110                     | +0.02283                     | 0                             |
| L1   | 0                            | +0.00118                     | 0                             |

The refined L1 run is the best available physical validation run. Its force history also provides the requested detailed view of drag and lift. The maximum drag is reached before the strongest lift excursion, and the final force values are close to zero because the ramped inlet velocity decays by  $t = 8$ .

### 6.1.5 V5 – Timestep Refinement on 2D-3

Setup:  $U_{\text{in}} = 1.5$  and  $Re = 100$  for all timesteps. Benchmark V5 studies the timestep dependence of the 2D-3 ramp result on mesh L0. Decreasing  $\Delta t$  improves the force extrema, especially the lift peak, and keeps the final pressure drop in the Schaefer–Turek interval for the smallest tested steps. Larger timesteps over-predict  $C_L^{\max}$  and can move  $\Delta p(8)$  outside the reference range. The timestep sweep is reported in Table 7, and the signed errors are listed in Table 8.

Table 7: V5 timestep-refinement results for the 2D-3 ramp case on L0.

| $\Delta t$          | $C_D^{\max}$  | $C_L^{\max}$  | $\Delta p(8)$      |
|---------------------|---------------|---------------|--------------------|
| $5 \cdot 10^{-4}$   | 2.92368       | 0.49769       | -0.11177           |
| $10^{-3}$           | 2.91949       | 0.51342       | -0.11069           |
| $2.5 \cdot 10^{-3}$ | 2.91169       | 0.56030       | -0.10722           |
| $5 \cdot 10^{-3}$   | 2.89917       | 0.63391       | -0.09869           |
| $10^{-2}$           | 2.89595       | 0.85780       | -0.11051           |
| Reference           | 2.9300–2.9700 | 0.4700–0.4900 | [−0.1150, −0.1050] |

Table 8: V5 signed error to the nearest Schaefer–Turek bound.

| $\Delta t$          | $e_{\partial I}(C_D^{\max})$ | $e_{\partial I}(C_L^{\max})$ | $e_{\partial I}(\Delta p(8))$ |
|---------------------|------------------------------|------------------------------|-------------------------------|
| $5 \cdot 10^{-4}$   | -0.00632                     | +0.00769                     | 0                             |
| $10^{-3}$           | -0.01051                     | +0.02342                     | 0                             |
| $2.5 \cdot 10^{-3}$ | -0.01831                     | +0.07030                     | 0                             |
| $5 \cdot 10^{-3}$   | -0.03083                     | +0.14391                     | +0.00631                      |
| $10^{-2}$           | -0.03405                     | +0.36780                     | 0                             |

The smallest timestep is the best temporal-discretization result in this sweep, but it still does not fully recover the refined L1 validation quality of V4.

### 6.1.6 V6 – Reynolds Sweep on 2D-3

Setup:  $U_{\text{in}} = 1.5$  with  $Re \in \{20, 100, 200\}$ . Benchmark V6 is a diagnostic Reynolds sweep with the same ramp-style setup. It is not a direct Schaefer–Turek validation case for all Reynolds numbers, but it shows how the solver reacts to viscosity changes. As  $Re$  increases, the lift response becomes much stronger and the divergence norm increases. The quantitative sweep is reported in Table 9. The drag and lift histories are shown in Figures 8 and 9, respectively, while Figures 10 and 11 show the pressure and velocity fields for the  $Re = 200$  case.

Table 9: V6 Reynolds sweep on the 2D-3 ramp setup.

| $Re$ | $C_D^{\max}$ | $C_L^{\max}$ | $\Delta p(8)$ | $\ \nabla \cdot u\ _{L^2}$ |
|------|--------------|--------------|---------------|----------------------------|
| 20   | 5.54948      | 0.02508      | -0.12760      | 0.02624                    |
| 100  | 2.91949      | 0.51342      | -0.11069      | 0.04274                    |
| 200  | 2.92042      | 1.81530      | -0.09089      | 0.07460                    |

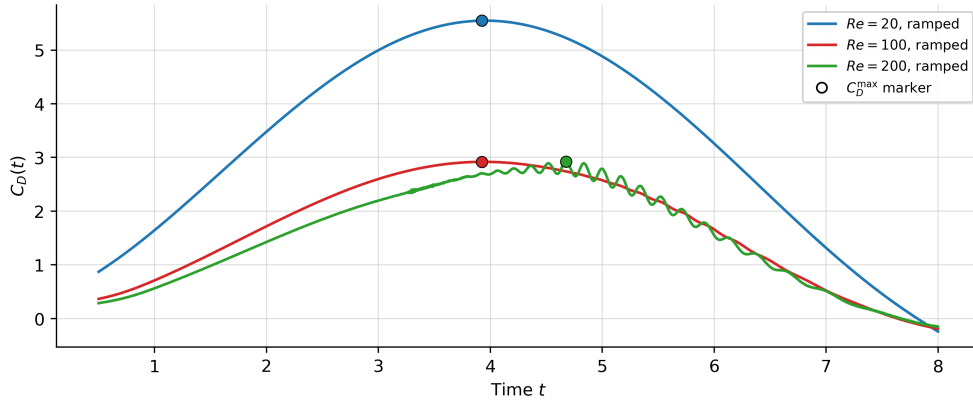


Figure 8: Time history of the drag coefficient  $C_D(t)$  for the Reynolds-number sweep on the 2D-3 ramp setup. The curves compare the responses at  $Re = 20$ ,  $Re = 100$ , and  $Re = 200$ , showing how the drag peak and its timing change as the Reynolds number increases. The markers indicate the maximum drag values used in the quantitative comparison.

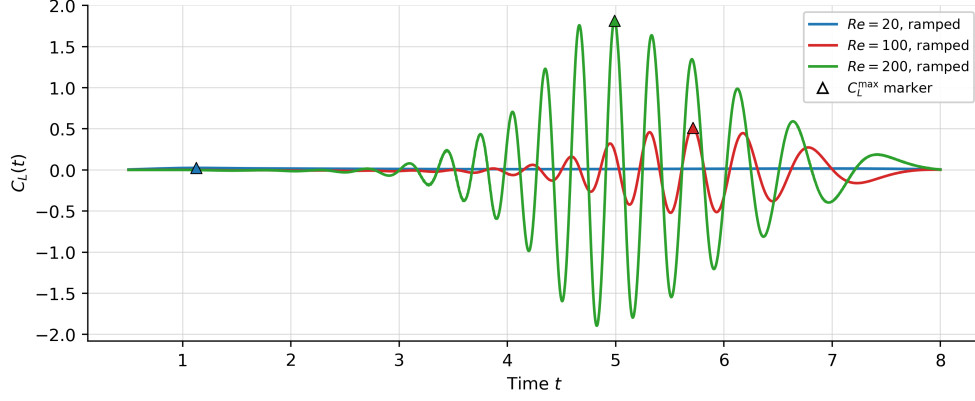


Figure 9: Time history of the lift coefficient  $C_L(t)$  for the Reynolds-number sweep on the 2D-3 ramp setup. The figure highlights the growth of unsteady lift oscillations with increasing Reynolds number, with the strongest response observed at  $Re = 200$ . The markers indicate the maximum lift values reported in the benchmark table.



Figure 10: Benchmark V6: pressure field at  $Re = 200$  and  $t = 4$  s.

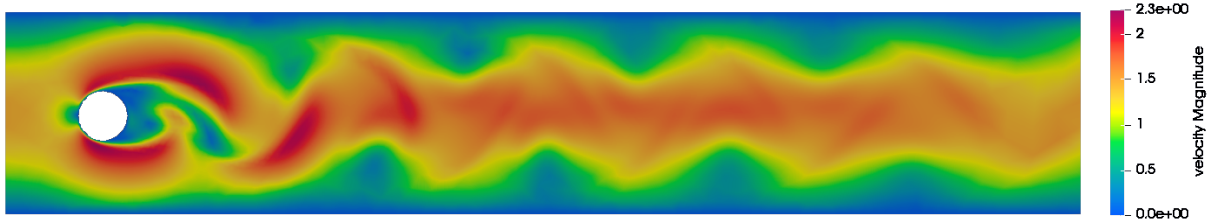


Figure 11: Benchmark V6: velocity magnitude at  $Re = 200$  and  $t = 4$  s.

### 6.1.7 V7 – Grad-Div Stabilization Sweep

Setup:  $U_{\text{in}} = 1.5$  and  $Re = 100$  for all stabilization values. Benchmark V7 isolates the effect of the grad-div parameter  $\gamma$ . The result is clear: increasing  $\gamma$  reduces the mean divergence norm substantially. It does not improve GMRES and increases the cost in this test; no performance benefit is observed. The best incompressibility is obtained with  $\gamma = 0.05$ , which is also the baseline used in the other final validation runs. The full sweep is summarized in Table 10.

Table 10: V7 grad-div sweep on the 2D-3 ramp setup.

| $\gamma$          | $C_D^{\max}$ | $C_L^{\max}$ | $\Delta p(8)$ | $\ \nabla \cdot u\ _{L^2}$ |
|-------------------|--------------|--------------|---------------|----------------------------|
| 0                 | 2.92667      | 0.50839      | -0.10405      | 0.23981                    |
| $10^{-3}$         | 2.92418      | 0.50428      | -0.11093      | 0.16499                    |
| $10^{-2}$         | 2.92933      | 0.50849      | -0.11222      | 0.07203                    |
| $5 \cdot 10^{-2}$ | 2.91949      | 0.51342      | -0.11069      | 0.04274                    |

### 6.1.8 V8 – 3D Validation

Setup:  $U_{\text{in}} = 2.25$ ,  $Re = 100$ , with inlet ramp  $T_{\text{ramp}} = 8$ . Benchmark V8 compares the 3D validation run on meshes  $3D\_L0$  and  $3D\_L1$ , with PCD and  $\Delta t = 10^{-3}$ . Both runs completed the full Schaefer–Turek 3D-3Z interval  $0 \leq t \leq 8$ . The L0 run uses 121,436 degrees of freedom; the L1 run uses 814,072 degrees of freedom. The inlet is the three-dimensional half-sine ramp with  $U_m = 2.25$ , so the instantaneous Reynolds number varies between 0 and 100.

The refined L1 mesh improves the 3D validation substantially: the drag maximum moves inside the Schaefer–Turek interval, and the pressure-drop error becomes small. The absolute lift maximum remains above the reference interval, but the overshoot is much smaller than on L0.

For the 3D-3Z benchmark the Schaefer–Turek paper reports intervals for  $C_D^{\max}$ ,  $C_L^{\max}$ , and  $\Delta p(8)$ . The same paper notes that, for 3D-3Z, the lift quantity should be interpreted as the maximum absolute lift coefficient. The reference comparison in Table 11 therefore uses  $|C_L|^{\max}$ ; the maximum positive signed lift is reported separately in Table 13.

Table 11: V8 comparison with the Schaefer–Turek 3D-3Z reference interval.

| Mesh      | DoFs    | $C_D^{\max}$  | $ C_L ^{\max}$ | $\Delta p(8)$      |
|-----------|---------|---------------|----------------|--------------------|
| $3D\_L0$  | 121,436 | 3.02849       | 0.02822        | -0.11305           |
| $3D\_L1$  | 814,072 | 3.23233       | 0.00584        | -0.11064           |
| Reference | –       | 3.2000–3.3000 | 0.0020–0.0040  | [−0.1100, −0.0900] |

Table 12: V8 signed error to the nearest Schaefer–Turek bound.

| Mesh     | $e_{\partial I}(C_D^{\max})$ | $e_{\partial I}( C_L ^{\max})$ | $e_{\partial I}(\Delta p(8))$ |
|----------|------------------------------|--------------------------------|-------------------------------|
| $3D\_L0$ | -0.17151                     | +0.02422                       | -0.00305                      |
| $3D\_L1$ | 0                            | +0.00184                       | -0.00064                      |

Table 13: V8 3D diagnostics over the completed interval  $0 \leq t \leq 8$ .

| Quantity                        | $3D\_L0$ | $3D\_L1$ |
|---------------------------------|----------|----------|
| $C_L^{\max}$ , signed positive  | 0.00117  | 0.00276  |
| $C_S^{\max}$                    | 0.00277  | 0.00004  |
| $\Delta p_{\max}$               | 3.29037  | 3.43039  |
| Mean $\ \nabla \cdot u\ _{L^2}$ | 0.35094  | 0.15225  |
| Max $\ \nabla \cdot u\ _{L^2}$  | 0.63798  | 0.27073  |

## 6.2 Preconditioner and Performance Benchmarks

The performance campaign uses the same two-dimensional cylinder geometry but is not intended as a physical validation run. The common setup is mesh L0,  $Re = 100$ ,  $\nu = 10^{-3}$ ,  $\Delta t = 10^{-3}$ , Oseen linearization with one nonlinear iteration per time step, outer GMRES tolerance  $10^{-6}$ , and four MPI ranks unless the benchmark explicitly sweeps a parameter. The runs stop at  $T = 1.5$ , and performance averages are computed only for  $t \geq 0.5$ , so the initial transient is excluded from the reported timing and iteration metrics.

All timing quantities are wall-clock times. *Runtime* is the total wall-clock time of the complete run. *Step time* is the mean total time per sampled time step for  $t \geq 0.5$ . *Setup time* is the mean time spent building the preconditioner per sampled time step. *Solve time* is the mean time spent in the outer FGMRES solve per sampled time step, including all applications of the preconditioner.

Deparis–Grandperrin–Quarteroni study PCD, Yosida, SIMPLE, and additive Schwarz preconditioners for linear systems arising from unsteady Navier–Stokes discretizations, with particular emphasis on strong scalability on large-scale parallel architectures [2]. The present report does not reproduce their benchmark problems, approximate parallel variants, meshes, or hardware. It only uses analogous qualitative criteria: efficiency is assessed through setup, solve, and time-to-solution; robustness through sensitivity to  $\Delta t$  and viscosity; optimality through mesh sensitivity; and scalability through the dependence of iteration count and runtime on the number of MPI ranks.

The performance runs were executed on a local workstation with an Intel Core i7-12700H CPU, 14 physical cores and 20 hardware threads, 16 GiB of memory as reported by the operating system, Ubuntu 24.04.4 LTS, Linux 6.8.0-124-generic, Open MPI 4.1.6, and  $g++$  13.3.0.

Table 14: Overview of the preconditioner and performance benchmarks.

| Benchmark | Case                               | Purpose                                                                                            |
|-----------|------------------------------------|----------------------------------------------------------------------------------------------------|
| P0        | Inner max-iteration tuning         | Chooses the caps for the inner preconditioner solves before the main performance comparison.       |
| P1        | Baseline preconditioner comparison | Compares PCD, SIMPLE, Yosida and block triangular at fixed $Re$ , $\Delta t$ , mesh and MPI ranks. |
| P2        | Timestep sensitivity               | Sweeps $\Delta t$ to assess iteration robustness and time-to-solution across preconditioners.      |
| P3        | Mesh sensitivity and optimality    | Compares L0 and L1 results to assess mesh dependence of the preconditioners.                       |
| P4        | Reynolds and viscosity sensitivity | Sweeps $Re$ to evaluate robustness with respect to the viscosity/flow parameter.                   |
| P5        | SIMPLE relaxation sweep            | Tunes the SIMPLE pressure relaxation parameter $\alpha$ and compares GMRES/runtime.                |
| P6        | Cost breakdown                     | Separates setup, solve, step and total runtime costs for each preconditioner.                      |
| P7        | MPI strong scaling                 | Sweeps 1, 2, 4 and 8 MPI ranks to report speedup, efficiency and iteration growth.                 |

### 6.2.1 P0 – Inner Max-Iteration Tuning

Benchmark P0 tunes the maximum number of iterations allowed in the inner preconditioner solves. The sweep uses  $m \in \{5, 10, 15, 20, 30, 50\}$ , applying the same cap to all inner



solves of each method: velocity and Schur for SIMPLE and block triangular, velocity and pressure for PCD, and velocity, Schur and correction for Yosida. All P0 runs have zero outer GMRES failures in the statistics window. The remaining distinction is therefore between the outer iteration plateau, the number of inner solves that still hit their cap, and the mean solve/step time.

Table 15: P0 inner max-iteration sweep. Bold rows are the selected caps.

| Preconditioner          | $m$       | GMRES mean/max  | Solve [s]     | Step [s]      |
|-------------------------|-----------|-----------------|---------------|---------------|
| PCD                     | 5         | 3.70 / 4        | 0.0145        | 0.0949        |
| PCD                     | 10        | 2.00 / 2        | 0.0138        | 0.0926        |
| PCD                     | 15        | 2.00 / 2        | 0.0150        | 0.0938        |
| PCD                     | 20        | 2.00 / 2        | 0.0150        | 0.0939        |
| <b>PCD</b>              | <b>30</b> | <b>2.00 / 2</b> | <b>0.0146</b> | <b>0.0920</b> |
| PCD                     | 50        | 2.00 / 2        | 0.0149        | 0.0934        |
| SIMPLE                  | 5         | 4.92 / 6        | 0.0207        | 0.0872        |
| <b>SIMPLE</b>           | <b>10</b> | <b>4.00 / 4</b> | <b>0.0277</b> | <b>0.0965</b> |
| SIMPLE                  | 15        | 4.00 / 4        | 0.0357        | 0.1057        |
| SIMPLE                  | 20        | 4.00 / 4        | 0.0369        | 0.1061        |
| SIMPLE                  | 30        | 4.00 / 4        | 0.0402        | 0.1101        |
| SIMPLE                  | 50        | 4.00 / 4        | 0.0474        | 0.1183        |
| Yosida                  | 5         | 6.59 / 11       | 0.0393        | 0.1050        |
| Yosida                  | 10        | 3.85 / 6        | 0.0404        | 0.1096        |
| Yosida                  | 15        | 3.30 / 6        | 0.0419        | 0.1129        |
| <b>Yosida</b>           | <b>20</b> | <b>3.27 / 6</b> | <b>0.0377</b> | <b>0.1033</b> |
| Yosida                  | 30        | 3.28 / 6        | 0.0382        | 0.1045        |
| Yosida                  | 50        | 3.29 / 6        | 0.0378        | 0.1053        |
| Block triangular        | 5         | 12.23 / 44      | 0.0412        | 0.1093        |
| Block triangular        | 10        | 7.77 / 31       | 0.0460        | 0.1133        |
| <b>Block triangular</b> | <b>15</b> | <b>5.31 / 8</b> | <b>0.0450</b> | <b>0.1133</b> |
| Block triangular        | 20        | 5.53 / 10       | 0.0505        | 0.1166        |
| Block triangular        | 30        | 5.53 / 10       | 0.0554        | 0.1235        |
| Block triangular        | 50        | 5.59 / 11       | 0.0621        | 0.1297        |

The selected caps are  $m = 30$  for PCD,  $m = 10$  for SIMPLE,  $m = 20$  for Yosida, and  $m = 15$  for block triangular.

### 6.2.2 P1 – Baseline Preconditioner Comparison

Benchmark P1 is the main fixed-parameter comparison at  $Re = 100$ ,  $\Delta t = 10^{-3}$ , mesh L0, and four MPI ranks. PCD is the most effective method in this configuration: it has the lowest outer GMRES count and the lowest linear solve time. SIMPLE remains competitive in total runtime because its setup/application costs are low, but it requires almost twice as many outer iterations. Yosida and block triangular are more expensive per step. The baseline comparison is summarized in Table 16.

Table 16: P1 baseline preconditioner comparison.

| Preconditioner   | GMRES mean/max | Setup [s] | Solve [s] | Step [s] | Runtime [s] |
|------------------|----------------|-----------|-----------|----------|-------------|
| PCD              | 2.30 / 3       | 0.0233    | 0.0176    | 0.1047   | 160.2       |
| SIMPLE           | 4.28 / 6       | 0.0224    | 0.0310    | 0.1056   | 164.0       |
| Yosida           | 4.12 / 8       | 0.0221    | 0.0478    | 0.1209   | 193.6       |
| Block triangular | 5.58 / 9       | 0.0223    | 0.0518    | 0.1247   | 191.0       |

### 6.2.3 P2 – Timestep Sensitivity

Benchmark P2 measures robustness with respect to  $\Delta t$ . All methods become harder to solve as the timestep increases, but the degradation is very different across preconditioners. PCD remains the most robust option in outer GMRES iterations over the full tested range. Time-to-solution gives a more nuanced picture because the number of time steps also changes with  $\Delta t$ : SIMPLE is faster for the largest tested time steps, while PCD keeps the most controlled iteration growth. Block triangular is especially sensitive for  $\Delta t = 10^{-2}$ , where the mean outer iteration count exceeds 700 and the runtime exceeds 1100 seconds. The iteration counts and total runtimes are reported in Tables 17 and 18, respectively.

Table 17: P2 mean GMRES iterations versus timestep.

| $\Delta t$          | PCD   | SIMPLE | Yosida | Block triangular |
|---------------------|-------|--------|--------|------------------|
| $5 \cdot 10^{-4}$   | 2.06  | 2.56   | 3.40   | 4.28             |
| $10^{-3}$           | 2.30  | 4.28   | 4.12   | 5.58             |
| $2.5 \cdot 10^{-3}$ | 4.33  | 9.46   | 8.71   | 21.17            |
| $5 \cdot 10^{-3}$   | 8.37  | 18.95  | 18.75  | 108.86           |
| $10^{-2}$           | 20.47 | 37.11  | 45.61  | 710.96           |

Table 18: P2 total runtime in seconds versus timestep.

| $\Delta t$          | PCD   | SIMPLE | Yosida | Block triangular |
|---------------------|-------|--------|--------|------------------|
| $5 \cdot 10^{-4}$   | 334.3 | 285.7  | 337.1  | 319.9            |
| $10^{-3}$           | 172.1 | 163.2  | 200.8  | 190.6            |
| $2.5 \cdot 10^{-3}$ | 81.9  | 87.3   | 162.6  | 172.0            |
| $5 \cdot 10^{-3}$   | 72.6  | 63.1   | 638.0  | 323.4            |
| $10^{-2}$           | 88.9  | 53.4   | 1847.5 | 1120.5           |

This behaviour is consistent with the qualitative lesson of the reference preconditioner paper: preconditioner quality must be evaluated using both iteration robustness and time to solution, not setup cost alone.

### 6.2.4 P3 – Mesh Sensitivity and Optimality

Benchmark P3 is the mesh-optimality test. Moving from L0 to L1 increases the problem size from 40,108 to 163,231 DoFs. PCD increases from 2.30 to 4.58 mean GMRES iterations, while block triangular shows only a mild increase from 5.58 to 5.77. Yosida remains

in the same low-iteration regime, with 4.12 mean iterations on L0 and 3.21 on L1. SIMPLE, in contrast, increases from 4.28 to 125.28 mean GMRES iterations. The completed data therefore indicate that PCD, Yosida, and block triangular are substantially closer to mesh-independent behaviour on this tested sequence, while SIMPLE loses robustness on the refined mesh. In wall-clock time, PCD and SIMPLE are the fastest methods on L0, whereas block triangular gives the lowest runtime on L1.

Table 19: P3 mesh-sensitivity results.

| Preconditioner   | Mesh | DoFs    | Mean GMRES | Solve [s] | Runtime [s] |
|------------------|------|---------|------------|-----------|-------------|
| PCD              | L0   | 40,108  | 2.30       | 0.0176    | 161.3       |
| PCD              | L1   | 163,231 | 4.58       | 0.4603    | 1298.5      |
| SIMPLE           | L0   | 40,108  | 4.28       | 0.0305    | 160.8       |
| SIMPLE           | L1   | 163,231 | 125.28     | 4.1464    | 6464.0      |
| Yosida           | L0   | 40,108  | 4.12       | 0.0479    | 195.5       |
| Yosida           | L1   | 163,231 | 3.21       | 0.3219    | 1209.8      |
| Block triangular | L0   | 40,108  | 5.58       | 0.0518    | 191.0       |
| Block triangular | L1   | 163,231 | 5.77       | 0.2764    | 881.5       |

### 6.2.5 P4 – Reynolds and Viscosity Sensitivity

Benchmark P4 measures robustness with respect to the physical parameter  $\nu$ , reported here through  $Re$ . Increasing  $Re$  increases the iteration count for all methods, but PCD remains below three mean GMRES iterations even at  $Re = 200$ . SIMPLE and Yosida show a moderate increase, and block triangular is consistently the least favourable in iteration count. The Reynolds sensitivity is summarized in Table 20.

Table 20: P4 mean GMRES iterations versus Reynolds number.

| $Re$ | PCD  | SIMPLE | Yosida | Block triangular |
|------|------|--------|--------|------------------|
| 20   | 1.79 | 2.31   | 2.82   | 4.09             |
| 100  | 2.30 | 4.28   | 4.12   | 5.58             |
| 200  | 2.85 | 5.70   | 5.51   | 7.14             |

### 6.2.6 P5 – SIMPLE Relaxation Sweep

Benchmark P5 tunes the SIMPLE pressure relaxation parameter  $\alpha$ . The best tested value is  $\alpha = 0.2$ , which gives the lowest mean GMRES count and the shortest runtime. Increasing  $\alpha$  steadily worsens the iteration count and solve time in this setup. The relaxation sweep is reported in Table 21.

Table 21: P5 SIMPLE relaxation sweep.

| $\alpha$ | Mean GMRES | Solve [s] | Step [s] | Runtime [s] |
|----------|------------|-----------|----------|-------------|
| 0.2      | 3.61       | 0.0247    | 0.0924   | 144.0       |
| 0.3      | 4.28       | 0.0281    | 0.0956   | 148.3       |
| 0.7      | 6.65       | 0.0414    | 0.1095   | 169.5       |
| 1.0      | 8.32       | 0.0495    | 0.1154   | 178.6       |

### 6.2.7 P6 – Cost Breakdown

Benchmark P6 separates assembly, preconditioner setup, and linear solve costs. The ranking is consistent with P1: PCD has the lowest solve time and total step time, SIMPLE is close in total runtime, and Yosida/block triangular are more expensive. The cost breakdown is reported in Table 22.

Table 22: P6 cost breakdown.

| Preconditioner   | Setup [s] | Solve [s] | Step [s] | Runtime [s] |
|------------------|-----------|-----------|----------|-------------|
| PCD              | 0.0200    | 0.0156    | 0.0905   | 138.0       |
| SIMPLE           | 0.0190    | 0.0270    | 0.0917   | 141.8       |
| Yosida           | 0.0188    | 0.0419    | 0.1067   | 171.5       |
| Block triangular | 0.0192    | 0.0458    | 0.1095   | 167.3       |

### 6.2.8 P7 – MPI Strong Scaling

Benchmark P7 measures strong scaling on the L0 problem with one, two, four, and eight MPI ranks. Table 23 reports the full sweep. All methods scale well up to four ranks, with speedups close to 3.1 and only mild iteration growth. Scaling saturates at eight ranks because the fixed problem has only 40,108 degrees of freedom, so the per-rank workload becomes too small and the parallel overhead dominates. PCD has the best eight-rank mean step time, while Yosida shows the largest iteration growth.

Table 23: P7 strong-scaling results for all tested MPI ranks.

| Preconditioner   | Ranks | Mean GMRES | Step [s] | Runtime [s] | Speedup | Eff. | Iter. growth |
|------------------|-------|------------|----------|-------------|---------|------|--------------|
| PCD              | 1     | 2.24       | 0.2804   | 427.1       | 1.00    | 1.00 | 1.00         |
| PCD              | 2     | 2.28       | 0.1558   | 237.5       | 1.80    | 0.90 | 1.02         |
| PCD              | 4     | 2.30       | 0.0931   | 138.3       | 3.09    | 0.77 | 1.03         |
| PCD              | 8     | 2.33       | 0.0921   | 140.6       | 3.04    | 0.38 | 1.04         |
| SIMPLE           | 1     | 4.17       | 0.2789   | 432.7       | 1.00    | 1.00 | 1.00         |
| SIMPLE           | 2     | 4.23       | 0.1557   | 241.1       | 1.79    | 0.90 | 1.01         |
| SIMPLE           | 4     | 4.28       | 0.0914   | 141.1       | 3.07    | 0.77 | 1.03         |
| SIMPLE           | 8     | 4.37       | 0.0940   | 146.0       | 2.96    | 0.37 | 1.05         |
| Yosida           | 1     | 3.81       | 0.3331   | 530.1       | 1.00    | 1.00 | 1.00         |
| Yosida           | 2     | 3.89       | 0.1846   | 292.0       | 1.82    | 0.91 | 1.02         |
| Yosida           | 4     | 4.12       | 0.1054   | 168.2       | 3.15    | 0.79 | 1.08         |
| Yosida           | 8     | 4.59       | 0.1184   | 188.8       | 2.81    | 0.35 | 1.21         |
| Block triangular | 1     | 5.53       | 0.3454   | 530.2       | 1.00    | 1.00 | 1.00         |
| Block triangular | 2     | 5.57       | 0.1920   | 293.8       | 1.80    | 0.90 | 1.01         |
| Block triangular | 4     | 5.58       | 0.1095   | 166.3       | 3.19    | 0.80 | 1.01         |
| Block triangular | 8     | 5.68       | 0.1142   | 175.7       | 3.02    | 0.38 | 1.03         |

## 6.3 Summary of Validation Status

The strongest physical validation result is V4 on the refined L1 mesh: the 2D-3 drag peak and final pressure drop are inside the Schaefer–Turek reference intervals, and the lift peak

---

is only marginally above the upper bound. V2 validates the shedding frequency through the Strouhal number, but its force amplitudes are less accurate. V1 is close in pressure drop but still misses the force intervals on L0.

For the preconditioner campaign, PCD is the most robust option overall. It has the lowest baseline GMRES count, the best timestep and Reynolds robustness, and the best mesh behaviour among the completed P3 runs. SIMPLE is competitive in runtime on the smallest mesh but is not mesh robust. Yosida is functional but more expensive in this implementation, and block triangular is the least robust with respect to large timesteps.

## 7 Conclusions

In this work, we developed and tested a finite element solver for the unsteady incompressible Navier–Stokes equations in two and three space dimensions. The solver is based on a mixed velocity–pressure formulation, using Taylor–Hood finite elements for the spatial discretization and a one-step  $\theta$ -method for time integration. The nonlinear convective term is treated through Oseen and Picard-type linearizations, while Temam, grad-div and SUPG stabilizations are included to improve robustness and control the effect of convection and discrete incompressibility errors.

The numerical validation was carried out on the classical flow-past-a-cylinder benchmark. In the two-dimensional cases, the solver was able to reproduce the main physical features of the benchmark, including the steady low-Reynolds-number regime and the periodic vortex-shedding regime. The comparison with the Schaefer–Turek reference data shows that mesh and timestep refinement improve the agreement of the drag coefficient, lift coefficient and pressure drop.

From the algebraic point of view, the discretization leads at each timestep to a large, sparse and nonsymmetric saddle-point system. For this reason, several block preconditioning strategies were implemented and compared: SIMPLE, Block Triangular, Yosida and Pressure Convection–Diffusion. The numerical experiments show that the choice of the preconditioner has a significant impact on both the number of outer FGMRES iterations and the total computational cost. Preconditioners based on Schur-complement approximations provide a practical way to exploit the block structure of the problem, while PCD gives a more pressure-operator-based approximation of the Navier–Stokes Schur complement and generally improves robustness on the tested configurations.

Overall, the project confirms that a stabilized Taylor–Hood finite element formulation, combined with suitable block preconditioning, provides an effective framework for the simulation of incompressible flows past a cylinder. The main limitations of the current work are related to the resolution of the finest benchmark cases, especially in three dimensions, and to the cost of the inner linear solves. Future improvements could include finer mesh refinement in the wake region, adaptive timestep strategies, more scalable parallel preconditioners and a more systematic validation of the three-dimensional case.

## References

- [1] N. Bootland, A. Bentley, and C. Kees. “Preconditioners for Two-Phase Incompressible Navier–Stokes Flow”. In: *SIAM Journal on Scientific Computing* 41 (2019), B843–B869.

- 
- [2] S. Deparis, G. Grandperrin, and A. Quarteroni. “Parallel preconditioners for the unsteady Navier–Stokes equations and applications to hemodynamics simulations”. In: *Computers & Fluids* 92 (2014), pp. 253–273.
  - [3] H. Elman and R. Tuminaro. “Boundary conditions in approximate commutator preconditioners for the Navier-Stokes equations”. In: *SIAM Journal on Scientific Computing* 35 (Jan. 2009), pp. 257–280.
  - [4] M. Schäfer et al. “Benchmark Computations of Laminar Flow Around a Cylinder”. In: *Vieweg+Teubner Verlag*. Wiesbaden, 1996, pp. 547–566.